DEVOPS ENGINEERING MASTERY

GIT, SCRIPTING & AUTOMATION ENGINEERING

Git, Bash, Python & Software Automation

VOLUME 03 • FIRST EDITION

```
nahid@ubuntu-server:~$ git log --oneline --graph
* a1b2c3d (HEAD -> main) Add health-check script
* 9f8e7d6 Merge feature/backup-automation
nahid@ubuntu-server:~$ ./health-check.sh
OK: disk 42% • memory 61% • nginx running
nahid@ubuntu-server:~$ python3 -m pytest tests/ -q
```

Ubuntu Server 24.04 LTS (Noble Numbat) — Primary hands-on environment

Git 2.43 • Bash 5.2 • Python 3.12 • pytest 9.2

First Edition • Published 8 November 2026 • Dhaka, Bangladesh

Companion Workbook • 24 Labs • 15 Assignments • 3 Projects

Practical Lab & Assignment Workbook

DEVOPS ENGINEERING MASTERY — Volume 03 (Companion Workbook). First Edition, 8 November 2026. Ubuntu Server 24.04 LTS, Git 2.43, Bash 5.2, Python 3.12. This workbook collects Appendix A (24 guided labs), Appendix B (15 independent assignments) and Appendix C (3 major projects) from the textbook, so you can work through them beside your virtual machine without carrying the whole book. Do the labs in order; each lab states its prerequisites. Never run a lab command on a production server. Solutions and marking guidance are in the separate Quiz & Answer Guide — attempt each assignment before looking.

পরিশিষ্ট A • Practical Labs — হাতে-কলমে অনুশীলন

Appendix A: ২৪টি Guided Practical Lab

এই পরিশিষ্টের প্রতিটি lab-ই বইয়ের অধ্যায়গুলোর বাস্তব প্রয়োগ। কোনো lab শুরু করার আগে সংশ্লিষ্ট অধ্যায়টি পড়ে নিন, তারপর ধাপে ধাপে নির্দেশনা অনুসরণ করুন। প্রতিটি lab-এ একটি বাস্তব দৃশ্য (real-world scenario), স্পষ্ট উদ্দেশ্য, ধাপে ধাপে নির্দেশনা, সম্পূর্ণ কোড, প্রত্যাশিত ফলাফল, যাচাই পদ্ধতি, সমস্যা সমাধান, নিরাপত্তা সতর্কতা ও cleanup আছে। সব কমান্ড Ubuntu Server 24.04 LTS, Git 2.43, Bash 5.2 ও Python 3.12-এ পরীক্ষিত। নমুনা আউটপুটগুলো **illustrative** — আপনার সিস্টেমে তারিখ, ব্যবহারকারীর নাম বা হ্যাশ আলাদা হতে পারে, কাঠামো একই থাকবে। কোনো lab-এই আসল পাসওয়ার্ড, টোকেন বা গোপন তথ্য ব্যবহার করবেন না; প্রয়োজনে পরিবেশ চলক (environment variable) বা প্লেসহোল্ডার ব্যবহার করুন।

> **CONCEPT:** Lab করার উদ্দেশ্য শুধু কমান্ড চালানো নয় — প্রতিটি কমান্ডের পেছনে কী ঘটছে তা বোঝা। কোনো ধাপ না বুঝলে থেমে অধ্যায়ে ফিরে যান, তারপর আবার চেষ্টা করুন। ভুল করা শেখার অংশ; cleanup ধাপটি তাই প্রতিটি lab-এ রাখা হয়েছে।

> **WARNING:** Lab-গুলো নিজের ল্যাপটপ, ভার্চুয়াল মেশিন বা পরীক্ষামূলক সার্ভারে করুন। Production সার্ভারে, অন্যের repository-তে বা অপরিচিত সার্ভারে এই কমান্ডগুলো চালাবেন না। `rm -rf`, `git reset --hard` ও `git push --force` ব্যবহারের আগে সবসময় কী মুছতে যাচ্ছেন তা যাচাই করুন।

Lab 01 — Git Repository তৈরি ও পরিদর্শন

বাস্তব দৃশ্য: আপনি TechNova-তে নতুন DevOps engineer হিসেবে যোগ দিয়েছেন। প্রথম কাজ — একটি ছোট automation প্রকল্পের জন্য স্থানীয় Git repository তৈরি করা, যাতে প্রতিটি পরিবর্তনের ইতিহাস সংরক্ষিত থাকে।

উদ্দেশ্য: `git init`, `git status`, `git add`, `git commit` ব্যবহার করে repository তৈরি করা এবং `.git` ডিরেক্টরির ভেতরের কাঠামো পরিদর্শন করা।

পূর্বজ্ঞান: Chapter 01–03; টার্মিনালে ফাইল তৈরি ও ডিরেক্টরি বদলের অভিজ্ঞতা। **পরিবেশ:** Ubuntu 24.04 LTS, Git 2.43, একটি খালি ডিরেক্টরি `~/labs/lab01`।

ধাপে ধাপে নির্দেশনা

1. পরীক্ষার ডিরেক্টরি তৈরি করুন ও তাতে ঢুকুন:

```
mkdir -p ~/labs/lab01
cd ~/labs/lab01
```

2. Git পরিচয় একবারের জন্য ঠিক করুন (শুধু এই মেশিনে, প্রথমবার):

```
git config --global user.name "Nahid Hasan"
git config --global user.email "nahid@example.com"
git config --global init.defaultBranch main
```

3. খালি repository তৈরি করুন:

```
git init
ls -la
```

4. একটি `README.md` ও একটি `hello.sh` ফাইল তৈরি করুন:

```
printf '# Lab 01 Project\n\nA small automation project.\n' > README.md
printf '#!/usr/bin/env bash\necho "Hello, Git!"\n' > hello.sh
chmod +x hello.sh
```

5. অবস্থা দেখুন, তারপর stage ও commit করুন:

```
git status
git add README.md hello.sh
git status
git commit -m "Add README and hello script"
git log --oneline
git status
```

6. `.git`-এর ভেতরটা পরিদর্শন করুন:

```
ls .git
cat .git/HEAD
ls .git/refs/heads
git branch
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`git init` বর্তমান ডিরেক্টরিতে একটি লুকানো `.git` ডিরেক্টরি তৈরি করে — এটাই repository। `git add` ফাইলকে working directory থেকে staging area-তে নেয়, অর্থাৎ পরের commit-এ কী যাবে তা ঠিক করে। `git commit` staging area-র একটি স্থায়ী snapshot তৈরি করে। `.git/HEAD` ফাইলে লেখা থাকে বর্তমান branch-এর নাম; সেটি একটি ref, যা সর্বশেষ commit-কে নির্দেশ করে।

প্রত্যাশিত ফলাফল (illustrative)

```
[main (root-commit) a1b2c3d] Add README and hello script
 2 files changed, 4 insertions(+)
 create mode 100644 README.md
 create mode 100755 hello.sh
```

`git log --oneline`-এ একটি লাইন দেখা যাবে; `cat .git/HEAD`-এ দেখা যাবে `ref: refs/heads/main`।

যাচাই

- `git status` বলবে `nothing to commit, working tree clean`।
- `git log`-এ আপনার নাম, ইমেইল ও বার্তা দেখা যাবে।
- `ls .git/objects`-এ নতুন object ডিরেক্টরি তৈরি হয়েছে।

> **TROUBLESHOOTING:** `Author identity unknown` এলে `git config --global user.name` ও `user.email` ঠিক করুন। `git commit`-এ কিছু না থাকলে আগে `git add` করেছেন কি না দেখুন।

> **WARNING:** আসল ইমেইল public repository-তে প্রকাশ করতে না চাইলে GitHub-এর noreply ইমেইল ব্যবহার করুন। কখনো পাসওয়ার্ড বা টোকেন commit করবেন না।

Cleanup: `cd ~ && rm -rf ~/labs/lab01` — শুধু এই lab-এর ডিরেক্টরিটিই মুছুন, অন্য কিছু নয়।

Lab 02 — Commit History ও Git Object পরিদর্শন (cat-file)

বাস্তব দৃশ্য: একটি production incident-এর পর আপনাকে খুঁজে বের করতে হবে কোন commit-এ একটি ভুল configuration ঢুকেছিল। `git log` শুধু সারাংশ দেয়; ভেতরের object-গুলো দেখলে বোঝা যায় Git আসলে কী সংরক্ষণ করে।

উদ্দেশ্য: blob, tree ও commit object চেনা; `git cat-file`, `git ls-tree` ও `git show` ব্যবহার করা।

পূর্বজ্ঞান: Lab 01, Chapter 02–03। **পরিবেশ:** `~/labs/lab02`-এ নতুন repository।

ধাপে ধাপে নির্দেশনা

1. Repository তৈরি করে তিনটি commit করুন:

```
mkdir -p ~/labs/lab02 && cd ~/labs/lab02
git init
printf 'version=1\n' > config.txt
git add config.txt
git commit -m "Add config version 1"
printf 'version=2\n' > config.txt
git add config.txt
git commit -m "Update config to version 2"
printf 'notes\n' > notes.txt
git add notes.txt
git commit -m "Add notes file"
```

2. ইতিহাস ও প্রতিটি commit-এর ভেতর দেখুন:

```
git log --oneline
git log --format=fuller
git show HEAD
git show HEAD~1
```

3. সর্বশেষ commit-এর object পরীক্ষা করুন:

```
git cat-file -p HEAD
git cat-file -t HEAD
git ls-tree HEAD
git cat-file -p HEAD^{tree}
```

4. একটি blob-এর বিষয়বস্তু বের করুন (ls-tree থেকে পাওয়া হ্যাশ বসান):

```
git cat-file -p <blob-hash>
git hash-object config.txt
git cat-file -t <blob-hash>
```

5. কোন commit-এ কোন লাইন বদলেছে তা দেখুন:

```
git log -p -- config.txt
git blame config.txt
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

প্রতিটি commit একটি object, যার ভেতরে থাকে tree-র হ্যাশ, parent commit-এর হ্যাশ, লেখক, তারিখ ও বার্তা। Tree object ডিরেক্টরির তালিকা ধারণ করে — কোন ফাইলের নাম, কোন blob হ্যাশ। Blob object শুধু ফাইলের বিষয়বস্তু ধারণ করে, নাম নয়। `git cat-file -p` কোনো object-এর বিষয়বস্তু মানুষের পড়ার মতো করে দেখায়, আর `-t` তার ধরন বলে। একই বিষয়বস্তুর ফাইল দুবার commit করলে একই blob হ্যাশ তৈরি হয় — এটাই Git-এর content-addressable storage।

প্রত্যাশিত ফলাফল (illustrative)

```
tree 9f86d0818...
parent 3a2c1e5...
author Nahid Hasan <nahid@example.com> 1762632000 +0600
committer Nahid Hasan <nahid@example.com> 1762632000 +0600

Add notes file
```

যাচাই

- `git log --oneline`-এ তিনটি commit দেখা যাবে।
- `git hash-object config.txt`-এর ফল `git ls-tree HEAD`-এর blob হ্যাশের সঙ্গে মিলবে।
- `git blame config.txt`-এ প্রতিটি লাইনের পাশে সঠিক commit দেখা যাবে।

> **TROUBLESHOOTING:** `fatal: Not a valid object name HEAD` মানে এখনো কোনো commit হয়নি — আগে commit করুন। `<blob-hash>` জায়গায় আসল হ্যাশ বসাতে হবে; `git ls-tree HEAD`-এর আউটপুট থেকে কপি করুন।

> **PRO TIP:** Incident তদন্তে `git log -p -- <file>` সবচেয়ে কার্যকর — কোন commit-এ কোন লাইন বদলেছে, এক নজরে দেখা যায়।

Cleanup: `cd ~ && rm -rf ~/labs/lab02`।

Lab 03 — Branching ও Merging

বাস্তব দৃশ্য: TechNova-র মূল সার্ভার-মনিটরিং স্ক্রিপ্টটি `main` branch-এ স্থিতিশীলভাবে চলছে। আপনি মূল কোড না ভেঙে একটি নতুন feature — disk-alert — আলাদা branch-এ তৈরি করবেন, পরীক্ষা করে main-এ merge করবেন।

উদ্দেশ্য: Branch তৈরি, বদল, merge ও branch মুছে ফেলা; fast-forward বনাম three-way merge বোঝা।

পূর্বজ্ঞান: Lab 01, Chapter 04। **পরিবেশ:** `~/labs/lab03`।

ধাপে ধাপে নির্দেশনা

1. ভিত্তি repository তৈরি করুন:

```
mkdir -p ~/labs/lab03 && cd ~/labs/lab03
git init
printf '#!/usr/bin/env bash\necho "monitor running"\n' > monitor.sh
chmod +x monitor.sh
git add monitor.sh
git commit -m "Add monitor script"
```

2. Feature branch তৈরি করে তাতে কাজ করুন:

```
git branch feature/disk-alert
git switch feature/disk-alert
printf '\n# Disk alert feature\necho "checking disk"\n' >> monitor.sh
git add monitor.sh
git commit -m "Add disk alert check"
git log --oneline --graph --all
```

3. main-এ ফিরে একটি আলাদা commit করুন (three-way merge তৈরি করতে):

```
git switch main
printf '# Server monitor\n' > README.md
git add README.md
git commit -m "Add README"
```

4. Feature branch merge করুন ও ইতিহাস দেখুন:

```
git merge feature/disk-alert
git log --oneline --graph --all
cat monitor.sh
```

5. Branch মুছে ফেলুন:

```
git branch -d feature/disk-alert
git branch
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

Branch হলো একটি commit-এর দিকে নির্দেশ করা হালকা pointer — নতুন branch তৈরি করতে Git কোনো ফাইল কপি করে না। `git switch` HEAD-কে অন্য branch-এ সরায় এবং working directory-র ফাইলগুলো সেই branch-এর অবস্থায় বদলে দেয়। Main-এ নতুন commit থাকায় merge-টি fast-forward হয়নি; Git দুটি parent-সহ একটি merge commit তৈরি করেছে — এটাই three-way merge। Merge-এর পর branch মুছে ফেলা নিরাপদ, কারণ commit-গুলো এখন main-এর ইতিহাসের অংশ।

প্রত্যাশিত ফলাফল (illustrative)

```
Merge made by the 'ort' strategy.
 monitor.sh | 3 +++
 1 file changed, 3 insertions(+)
```

গ্রাফে main ও feature branch-এর commit-গুলো মিলে যাওয়া দেখা যাবে।

যাচাই

- `git log --oneline --graph --all`-এ merge commit-এর দুটি parent লাইন দেখা যাবে।
- `cat monitor.sh`-এ disk-alert লাইনটি থাকবে।
- `git branch`-এ শুধু `main` থাকবে।

> **TROUBLESHOOTING:** Merge-এর সময় `error: Your local changes would be overwritten` এলে আগে `git stash` বা `git commit` করে working tree পরিষ্কার করুন। ভুল branch-এ commit করে ফেললে `git log` দেখে শান্তভাবে `git cherry-pick` (Lab 05) ব্যবহার করুন।

> **WARNING:** Public branch-এ push করা commit-এর ইতিহাস বদলাবেন না; প্রয়োজনে `git revert` ব্যবহার করুন।

Cleanup: `cd ~ && rm -rf ~/labs/lab03`।

Lab 04 — Merge Conflict সমাধান

বাস্তব দৃশ্য: আপনি ও আপনার সহকর্মী রাফি একই configuration ফাইলের একই লাইন বদলেছেন — আপনি timeout বাড়িয়েছেন, রাফি কমিয়েছেন। Merge করার সময় Git সিদ্ধান্ত নিতে পারবে না; মানুষ হিসেবে আপনাকেই সঠিক মানটি বেছে নিতে হবে।

উদ্দেশ্য: ইচ্ছাকৃতভাবে conflict তৈরি করা, conflict marker পড়া, হাতে সমাধান করা ও merge সম্পূর্ণ করা।

পূর্বজ্ঞান: Lab 03, Chapter 04। **পরিবেশ:** `~/labs/lab04`।

ধাপে ধাপে নির্দেশনা

1. ভিত্তি ফাইলসহ repository তৈরি করুন:

```
mkdir -p ~/labs/lab04 && cd ~/labs/lab04
git init
printf 'timeout=30\nretries=3\n' > app.conf
git add app.conf
git commit -m "Add app config"
```

2. দুটি branch-এ একই লাইন আলাদাভাবে বদলান:

```
git switch -c feature/timeout-60
printf 'timeout=60\nretries=3\n' > app.conf
git add app.conf
git commit -m "Increase timeout to 60"
git switch main
printf 'timeout=10\nretries=3\n' > app.conf
git add app.conf
git commit -m "Decrease timeout to 10"
```

3. Merge করে conflict দেখুন:

```
git merge feature/timeout-60
git status
cat app.conf
```

4. ফাইলটি সম্পাদনা করে conflict marker সরান — সিদ্ধান্ত নিন `timeout=45` আপস-মান হিসেবে রাখবেন:

```
printf 'timeout=45\nretries=3\n' > app.conf
git add app.conf
git commit -m "Resolve timeout conflict: use 45 seconds"
```

5. ফল যাচাই করুন:

```
git log --oneline --graph --all
cat app.conf
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

Conflict marker-এর `<<<<<<< HEAD` অংশটি আপনার বর্তমান branch-এর (main) সংস্করণ, `=======`-এর নিচের অংশটি incoming branch-এর সংস্করণ, আর `>>>>>>>` তার শেষ। Git ফাইলটিকে মাঝপথে রেখে দেয় — সমাধান না হওয়া পর্যন্ত merge সম্পূর্ণ হয় না। `git add` এখানে নতুন অর্থ বহন করে: এটি Git-কে জানায় এই ফাইলটির conflict সমাধান হয়েছে। শুধু marker মুছে ফেললেই হবে না; সঠিক মানটি নিয়ে দলের সঙ্গে আলোচনা করাই আসল সমাধান — এই lab-এ ৪৫ সেকেন্ড একটি যুক্তিসংগত আপস।

প্রত্যাশিত ফলাফল (illustrative)

```
Auto-merging app.conf
CONFLICT (content): Merge conflict in app.conf
Automatic merge failed; fix conflicts and then commit the result.
```

সমাধানের পর `cat app.conf`-এ `timeout=45` দেখা যাবে এবং কোনো marker থাকবে না।

যাচাই

- `grep -r "<<<<<<<" .` কোনো ফল দেবে না — কোনো marker অবশিষ্ট নেই।
- `git status` বলবে working tree clean।
- `git log --graph`-এ merge commit দেখা যাবে।

> **TROUBLESHOOTING:** Merge বাতিল করতে চাইলে সমাধানের আগে `git merge --abort` চালান। ভুল সমাধান commit করে ফেললে নতুন commit দিয়ে ঠিক করুন; public ইতিহাসে `reset --hard` করবেন না।

> **WARNING:** Conflict সমাধানের সময় দুই পাশের কোড না বুঝে এক পাশ মুছে ফেলবেন না — সহকর্মীর গুরুত্বপূর্ণ পরিবর্তন হারিয়ে যেতে পারে। সন্দেহ হলে `git log` দেখে তার সঙ্গে কথা বলুন।

Cleanup: `cd ~ && rm -rf ~/labs/lab04`।

Lab 05 — Rebase ও Cherry-pick

বাস্তব দৃশ্য: আপনার feature branch-এ তিনটি ছোট commit আছে, কিন্তু main এগিয়ে গেছে। পরিষ্কার, রৈখিক ইতিহাস রাখতে আপনি branch-টিকে main-এর উপর rebase করবেন; আর একটি জরুরি bug-fix commit শুধু আলাদা করে অন্য branch-এ নিতে cherry-pick ব্যবহার করবেন।

উদ্দেশ্য: `git rebase` দিয়ে ইতিহাস সাজানো, rebase conflict সামলানো ও `git cherry-pick` ব্যবহার করা।

পূর্বজ্ঞান: Lab 03–04, Chapter 04। **পরিবেশ:** `~/labs/lab05`।

ধাপে ধাপে নির্দেশনা

1. Repository ও feature branch তৈরি করুন:

```
mkdir -p ~/labs/lab05 && cd ~/labs/lab05
git init
printf 'v1\n' > app.txt
git add app.txt
git commit -m "Initial version"
git switch -c feature/report
printf 'v1\nreport: on\n' > app.txt
git add app.txt
git commit -m "Enable report"
printf 'v1\nreport: on\nformat: json\n' > app.txt
git add app.txt
git commit -m "Set report format to JSON"
```

2. main এগিয়ে নিন, তারপর feature rebase করুন:

```
git switch main
printf 'v2\n' > app.txt
git add app.txt
git commit -m "Bump to version 2"
git switch feature/report
git rebase main
git log --oneline --graph --all
```

3. একটি bug-fix commit cherry-pick করুন:

```
git switch main
printf 'fix: timeout\n' > fix.txt
git add fix.txt
git commit -m "Fix timeout bug"
git switch feature/report
git cherry-pick main
git log --oneline --graph --all
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

Rebase আপনার commit-গুলোকে main-এর সর্বশেষ অবস্থার উপর নতুন করে প্রয়োগ করে — পুরনো commit-গুলোর নতুন হ্যাশ তৈরি হয়, তাই ইতিহাস রৈখিক দেখায় কিন্তু আসলে commit পুনর্লেখন হয়। এ কারণে শুধু নিজের স্থানীয়, এখনো push না করা branch-এ rebase করা নিরাপদ। Cherry-pick একটি নির্দিষ্ট commit-এর পরিবর্তন অন্য branch-এ কপি করে; মূল commit অপরিবর্তিত থাকে, নতুন branch-এ নতুন হ্যাশসহ একই পরিবর্তন আসে। Rebase-এর সময় conflict হলে ফাইল ঠিক করে `git add` দিয়ে `git rebase --continue` চালাতে হয়; বাতিল করতে `git rebase --abort`।

প্রত্যাশিত ফলাফল (illustrative)

```
Successfully rebased and updated refs/heads/feature/report.
```

গ্রাফে feature branch-এর commit-গুলো main-এর commit-এর ঠিক পরে সরল রেখায় দেখা যাবে।

যাচাই

- `git log --oneline feature/report`-এ commit-গুলোর ক্রম হবে: Bump to version 2, Enable report, Set report format, Fix timeout bug।
- `cat app.txt` ও `cat fix.txt` প্রত্যাশিত বিষয়বস্তু দেখাবে।

> **TROUBLESHOOTING:** Rebase থেমে গেলে `git status` দেখুন — কোন ফাইলে conflict তা বলবে। সমাধান না করে এগোবেন না। ভুল rebase হলে `git reflog` দিয়ে আগের অবস্থা খুঁজে `git reset --hard <commit>` করতে পারেন — তবে শুধু নিজের স্থানীয় branch-এ।

> **WARNING:** অন্যের সঙ্গে ভাগ করা (pushed) branch-এ কখনো rebase করে force-push করবেন না; তাতে সহকর্মীদের কাজ নষ্ট হয়। সন্দেহ হলে merge ব্যবহার করুন।

Cleanup: `cd ~ && rm -rf ~/labs/lab05`।

Lab 06 — GitHub Collaboration: Fork, Clone ও Pull Request

বাস্তব দৃশ্য: TechNova-র একটি open-source টুলে আপনি একটি ছোট সংশোধন (README-তে একটি ভুল বানান ঠিক করা) জমা দিতে চান। আপনার নিজের সরাসরি লেখার অনুমতি নেই — তাই fork করে, clone করে, branch-এ কাজ করে pull request পাঠাতে হবে।

উদ্দেশ্য: GitHub-এ fork, SSH clone, feature branch, push ও pull request-এর সম্পূর্ণ প্রবাহ হাতে-কলমে করা।

পূর্বজ্ঞান: Lab 01–03, Chapter 05–06; একটি GitHub অ্যাকাউন্ট ও SSH key। **পরিবেশ:** ব্রাউজার, টার্মিনাল, ইন্টারনেট সংযোগ।

ধাপে ধাপে নির্দেশনা

1. SSH key আছে কি না যাচাই করুন; না থাকলে তৈরি করুন (passphrase দিন):

```
ls ~/.ssh/id_ed25519.pub
ssh-keygen -t ed25519 -C "nahid@example.com"
cat ~/.ssh/id_ed25519.pub
```

2. Public key-টি GitHub-এ যোগ করুন: GitHub → Settings → SSH and GPG keys → New SSH key। তারপর সংযোগ পরীক্ষা করুন:

```
ssh -T git@github.com
```

3. GitHub-এ একটি ছোট public repository বেছে Fork করুন (উদাহরণ: আপনার নিজের বা পরিচিত কোনো ছোট প্রকল্প), তারপর নিজের fork-টি clone করুন:

```
git clone git@github.com:YOUR-USERNAME/REPO-NAME.git
cd REPO-NAME
git remote -v
```

4. মূল repository-কে upstream হিসেবে যোগ করুন ও feature branch তৈরি করুন:

```
git remote add upstream git@github.com:ORIGINAL-OWNER/REPO-NAME.git
git fetch upstream
git switch -c fix/readme-typo
```

5. একটি ছোট পরিবর্তন করুন, commit করে নিজের fork-এ push করুন:

```
printf '\nFixed a typo in the documentation.\n' >> README.md
git add README.md
git commit -m "Fix typo in README"
git push origin fix/readme-typo
```

6. GitHub ওয়েবসাইটে গিয়ে Compare & pull request-এ ক্লিক করুন। শিরোনামে কী বদলেছেন, বর্ণনায় কেন বদলেছেন — দুটোই লিখুন। তারপর Submit করুন।
7. Maintainer-এর মন্তব্য এলে জবাব দিন; merge হয়ে গেলে স্থানীয় main হালনাগাদ করুন:

```
git switch main
git pull upstream main
```

```
git push origin main
git branch -d fix/readme-typo
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`origin` হলো আপনার fork-এর remote নাম, `upstream` মূল প্রকল্পের। এই আলাদা নামকরণই fork-ভিত্তিক workflow-এর মূল — আপনি upstream থেকে নতুন পরিবর্তন আনেন (`fetch` / `pull`), কিন্তু push করেন শুধু নিজের fork-এ। Pull request কোনো Git কমান্ড নয়, এটি GitHub-এর একটি পর্যালোচনা প্রক্রিয়া: আপনি maintainer-কে বলছেন, আমার branch-এর পরিবর্তনগুলো দেখে আপনার main-এ নিন কি না সিদ্ধান্ত নিন।

প্রত্যাশিত ফলাফল (illustrative)

```
Hi Nahid! You've successfully authenticated, but GitHub does not provide shell access.
```

Pull request পৃষ্ঠায় `Able to merge` ও আপনার একটি commit দেখা যাবে।

যাচাই

- `git remote -v`-এ origin ও upstream দুটোই দেখা যাবে।
- GitHub-এ আপনার fork-এর branch-টি ও pull request নম্বর দেখা যাবে।
- Merge-এর পর upstream main-এ আপনার পরিবর্তনটি দেখা যাবে।

TROUBLESHOOTING: `Permission denied (publickey)` এলে SSH key GitHub-এ যোগ হয়েছে কি না এবং `ssh-add ~/.ssh/id_ed25519` চালানো হয়েছে কি না দেখুন। `git push`-এ rejected হলে আগে `git pull --rebase upstream main` করে নিন।

WARNING: Private key (`~/.ssh/id_ed25519`) কখনো কাউকে দেবেন না, কোথাও আপলোড করবেন না, commit করবেন না। শুধু `.pub` ফাইলটি শেয়ার করা নিরাপদ।

Cleanup: পরীক্ষার fork-টি GitHub-এ Delete repository করে মুছে ফেলুন; স্থানীয় clone ডিরেক্টরি মুছুন।

Lab 07 — Bash Variables ও User Input

বাস্তব দৃশ্য: প্রতিদিন সকালে অপারেটরকে সার্ভারের নাম ও একটি সতর্কবার্তা টাইপ করে একটি স্বাগত বার্তা তৈরি করতে হয়। হাতে লিখে ভুল হয়; একটি ছোট স্ক্রিপ্ট ব্যবহারকারীর কাছ থেকে মান নিয়ে নিরাপদে বার্তা তৈরি করবে।

উদ্দেশ্য: Variable ঘোষণা, `read` দিয়ে input নেওয়া, quoting, command substitution ও positional parameter ব্যবহার করা।

পূর্বজ্ঞান: Chapter 07–08। **পরিবেশ:** `~/labs/lab07`, Bash 5.2।

ধাপে ধাপে নির্দেশনা

1. ডিরেক্টরি তৈরি করুন ও স্ক্রিপ্টটি লিখুন:

```
mkdir -p ~/labs/lab07 && cd ~/labs/lab07
cat > greet.sh <<'EOF'
```

```
#!/usr/bin/env bash
set -u
server_name="${1:-web-01}"
read -r -p "Your name: " operator_name
read -r -s -p "API key (will not echo): " api_key
printf '\n'
today="$(date +%Y-%m-%d)"
printf 'Hello %s, server %s checked on %s.\n' "$operator_name" "$server_name" "$today"
printf 'API key length: %s characters.\n' "${#api_key}"
EOF
chmod +x greet.sh
```

2. স্বাভাবিকভাবে ও argument দিয়ে চালান:

```
./greet.sh
./greet.sh db-01
```

3. খালি input দিয়ে পরীক্ষা করুন — কী হয় দেখুন:

```
./greet.sh ""
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`${1:-web-01}` মানে প্রথম argument না দিলে `web-01` ব্যবহার করো — এটি একটি নিরাপদ default। `read -r` backslash-কে আক্ষরিকভাবে পড়ে; `-p` prompt দেখায়; `-s` টাইপ করা অক্ষর পর্দায় দেখায় না, তাই গোপন মানের জন্য উপযুক্ত। `"$(date +%Y-%m-%d)"` হলো command substitution — কমান্ডের আউটপুট variable-এ ধরা হয়। সব variable ব্যবহারের সময় double quote (`"$operator_name"`) ব্যবহার করা হয়েছে, যাতে নামে space থাকলেও তা একটিই মান হিসেবে থাকে। `${#api_key}` key-র দৈর্ঘ্য দেয়, কিন্তু key-টি কখনো print করে না — এটাই নিরাপদ অভ্যাস।

প্রত্যাশিত ফলাফল (illustrative)

```
Your name: Nahid
API key (will not echo):
Hello Nahid, server web-01 checked on 2026-11-08.
API key length: 12 characters.
```

যাচাই

- নামে space দিয়ে (যেমন `Nahid Hasan`) চালালে পুরো নাম এক লাইনে দেখা যাবে।
- `bash -n greet.sh` কোনো syntax error দেবে না।
- `shellcheck greet.sh` কোনো সতর্কতা দিলে তা ঠিক করুন (ShellCheck না থাকলে `sudo apt install shellcheck`)।

> **TROUBLESHOOTING:** `unbound variable` এলে কোনো variable ঘোষণার আগেই ব্যবহার করা হয়েছে — `set -u` সেটাই ধরেছে। `read` -এ backslash হারিয়ে গেলে `-r` যোগ করুন।

> **WARNING:** গোপন মান কখনো `echo` দিয়ে print করবেন না, log ফাইলে লিখবেন না, বা command-line argument হিসেবে পাস করবেন না — argument অন্য ব্যবহারকারী `ps` দিয়ে দেখতে পারে।

Cleanup: `cd ~ && rm -rf ~/labs/lab07`।

Lab 08 — Bash Conditions ও Loops

বাস্তব দৃশ্য: TechNova-র একটি ডিরেক্টরিতে প্রতিদিন শত শত log ফাইল জমে। ৭ দিনের পুরনো ফাইলগুলো archive করতে হয়, আর disk ব্যবহার ৮০%-এর বেশি হলে সতর্কবার্তা দিতে হয়। হাতে করলে ভুল হবে; condition ও loop দিয়ে স্বয়ংক্রিয় করুন।

উদ্দেশ্য: `if/elif/else`, `test`, `for` ও `while` loop, সংখ্যা ও স্ট্রিং তুলনা ব্যবহার করা।

পূর্বজ্ঞান: Lab 07, Chapter 09। **পরিবেশ:** `~/labs/lab08`।

ধাপে ধাপে নির্দেশনা

1. পরীক্ষার log ফাইল তৈরি করুন (কিছু পুরনো তারিখসহ):

```
mkdir -p ~/labs/lab08/logs ~/labs/lab08/archive
cd ~/labs/lab08
touch logs/app-2026-11-01.log logs/app-2026-11-07.log logs/app-2026-11-08.log
touch -d "10 days ago" logs/app-2026-11-01.log
touch -d "2 days ago" logs/app-2026-11-07.log
```

2. Archive স্ক্রিপ্টটি লিখুন:

```
cat > archive_old_logs.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
log_dir="$HOME/labs/lab08/logs"
archive_dir="$HOME/labs/lab08/archive"
if [[ ! -d "$log_dir" ]]; then
  printf 'Error: log directory %s not found.\n' "$log_dir" >&2
  exit 1
fi
count=0
for file in "$log_dir"/*.log; do
  if [[ ! -e "$file" ]]; then
    printf 'No log files found.\n'
    break
  fi
  if [[ "$file" -ot "$archive_dir" ]] || find "$file" -mtime +7 -print -quit | grep -q .;
then
    mv -- "$file" "$archive_dir/"
    printf 'Archived: %s\n' "$(basename "$file")"
    count=$((count + 1))
  else
    printf 'Keeping: %s\n' "$(basename "$file")"
  fi
done
printf 'Total archived: %d\n' "$count"
usage="$(df --output=pcent / | tail -n 1 | tr -dc '0-9')"
```

```
if [[ "$usage" -ge 80 ]]; then
  printf 'WARNING: disk usage is %s%%.\n' "$usage" >&2
elif [[ "$usage" -ge 60 ]]; then
  printf 'Notice: disk usage is %s%%.\n' "$usage"
else
  printf 'Disk usage OK: %s%%.\n' "$usage"
fi
EOF
chmod +x archive_old_logs.sh
```

3. চালান ও ফল দেখুন:

```
./archive_old_logs.sh
ls logs archive
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`[[ ! -d "$log_dir" ]]` ডিরেক্টরি আছে কি না পরীক্ষা করে; না থাকলে স্ক্রিপ্টটি error বার্তা দিয়ে exit code 1 নিয়ে থামে। `for file in "$log_dir"/*.log` প্রতিটি log ফাইলে একবার করে loop ঘোরে — ফাইলের নামে space থাকলেও quote-এর কারণে ভাঙে না। `find "$file" -mtime +7` ফাইলটি ৭ দিনের পুরনো কি না জানায়। সংখ্যা তুলনায় `-ge` (greater or equal) ব্যবহার করা হয়েছে — স্ট্রিং তুলনার `>` এখানে ভুল ফল দিত। `count=$((count + 1))` arithmetic expansion-এর নিরাপদ রূপ।

প্রত্যাশিত ফলাফল (illustrative)

```
Archived: app-2026-11-01.log
Keeping: app-2026-11-07.log
Keeping: app-2026-11-08.log
Total archived: 1
Disk usage OK: 42%.
```

যাচাই

- `ls archive`-এ শুধু ১০ দিনের পুরনো ফাইলটি থাকবে।
- আবার চালালে `Total archived: 0` হবে — স্ক্রিপ্টটি idempotent।
- Disk শতাংশ আপনার সিস্টেমের আসল মানের সঙ্গে মিলবে (`df -h /` দিয়ে মেলান)।

> **TROUBLESHOOTING:** `*.log` মিল না পেলে loop-এ আক্ষরিক `*.log` স্ট্রিং আসে — তাই `[[ ! -e "$file" ]]` পরীক্ষাটি রাখা হয়েছে। `integer expression expected` এলে `usage` variable-এ শুধু সংখ্যা আসছে কি না দেখুন (`%` চিহ্ন সরানো হয়েছে কি না)।

> **PRO TIP:** Condition-এ সবসময় `[[ ... ]]` ব্যবহার করুন — এটি `[ ... ]`-এর চেয়ে নিরাপদ, word splitting ও glob সমস্যা কম হয়।

Cleanup: `cd ~ && rm -rf ~/labs/lab08`।

Lab 09 — Functions ও Reusable Scripts

বাস্তব দৃশ্য: আপনার কাছে তিনটি আলাদা স্ক্রিপ্টে একই কাজ বারবার লেখা আছে — log বার্তা লেখা, ফাইল আছে কি না পরীক্ষা করা, ডিরেক্টরি তৈরি করা। একটি shared library ফাইলে function-গুলো একবার লিখে সব স্ক্রিপ্ট থেকে ব্যবহার করবেন।

উদ্দেশ্য: Function সংজ্ঞা, parameter, return status, `local` variable ও library sourcing শেখা।

পূর্বজ্ঞান: Lab 07–08, Chapter 10। **পরিবেশ:** `~/labs/lab09`।

ধাপে ধাপে নির্দেশনা

1. Library ফাইলটি তৈরি করুন:

```
mkdir -p ~/labs/lab09 && cd ~/labs/lab09
cat > lib.sh <<'EOF'
#!/usr/bin/env bash
log_info() {
  local message="$1"
  printf '[INFO] %s %s\n' "$(date '+%Y-%m-%d %H:%M:%S')" "$message"
}
log_error() {
  local message="$1"
  printf '[ERROR] %s %s\n' "$(date '+%Y-%m-%d %H:%M:%S')" "$message" >&2
}
ensure_dir() {
  local dir="$1"
  if [[ -z "$dir" ]]; then
    log_error "ensure_dir called with an empty path."
    return 1
  fi
  if [[ ! -d "$dir" ]]; then
    mkdir -p -- "$dir"
  fi
}
check_file() {
  local file="$1"
  if [[ -f "$file" ]]; then
    return 0
  fi
  log_error "File not found: $file"
  return 1
}
EOF
```

2. Library ব্যবহার করে মূল স্ক্রিপ্ট লিখুন:

```
cat > backup.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
script_dir="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
source "$script_dir/lib.sh"
source_file="${1:-}"
if [[ -z "$source_file" ]]; then
  log_error "Usage: backup.sh <file>"
  exit 1
```

```
fi
if ! check_file "$source_file"; then
  exit 1
fi
backup_dir="$HOME/labs/lab09/backups"
ensure_dir "$backup_dir"
cp -- "$source_file" "$backup_dir/$(basename "$source_file").bak"
log_info "Backed up $source_file"
EOF
chmod +x backup.sh
```

3. পরীক্ষা করুন:

```
printf 'important data\n' > data.txt
./backup.sh data.txt
./backup.sh missing.txt; echo "exit code: $?"
ls backups
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`local message="$1"` function-এর প্রথম argument-কে একটি স্থানীয় variable-এ ধরে — `local` না দিলে variable-টি global হয়ে যেত এবং অন্য function-এর মান নষ্ট করতে পারত। Function কোনো মান return করে না, return করে একটি status (0 = সফল, অন্য কিছু = ব্যর্থ); তাই `check_file` সফল হলে `return 0`, ব্যর্থ হলে `return 1` দেয়, আর কলার `if ! check_file ...` দিয়ে তা পরীক্ষা করে। `source "$script_dir/lib.sh"` library-কে বর্তমান shell-এ লোড করে — `./lib.sh` চালালে আলাদা process হতো, function পাওয়া যেত না। `BASH_SOURCE[0]` দিয়ে স্ক্রিপ্টটি যেখান থেকে চালানো হোক না কেন, তার নিজের ডিরেক্টরি বের করা হয়েছে।

প্রত্যাশিত ফলাফল (illustrative)

```
[INFO] 2026-11-08 10:00:00 Backed up data.txt
[ERROR] 2026-11-08 10:00:01 File not found: missing.txt
exit code: 1
```

যাচাই

- `ls backups`-এ `data.txt.bak` থাকবে, ভেতরে `important data` লেখা থাকবে।
- Argument ছাড়া চালালে usage বার্তা ও exit code 1 আসবে।
- দ্বিতীয়বার চালালেও একই ফল হবে — কোনো duplicate তৈরি হবে না।

TROUBLESHOOTING: `lib.sh: No such file or directory` এলে স্ক্রিপ্টটি অন্য ডিরেক্টরি থেকে চালাচ্ছেন — `script_dir` হিসাবটি ঠিক আছে কি না দেখুন। `command not found: log_info` মানে source লাইনটি চলেনি বা library-তে syntax error আছে — `bash -n lib.sh` চালান।

PRO TIP: Library ফাইলে `set -e` রাখবেন না — library যেকোনো স্ক্রিপ্ট থেকে source হতে পারে, তার নিজের সেটিং কলারের আচরণ বদলে দিতে পারে।

Cleanup: `cd ~ && rm -rf ~/labs/lab09`।

Lab 10 — Text ও Log Processing (grep, sed, awk)

বাস্তব দৃশ্য: একটি web সার্ভারের access log-এ হাজার হাজার লাইন। আপনাকে বের করতে হবে কোন IP সবচেয়ে বেশি request করেছে, কতগুলো 500 error হয়েছে, আর কোন URL-গুলো সবচেয়ে জনপ্রিয়। পুরো ফাইল হাতে পড়া অসম্ভব — pipeline দিয়ে বিশ্লেষণ করুন।

উদ্দেশ্য: `grep`, `sed`, `awk`, `sort`, `uniq`, `cut` দিয়ে log বিশ্লেষণ করা।

পূর্বজ্ঞান: Chapter 11। **পরিবেশ:** `~/labs/lab10`।

ধাপে ধাপে নির্দেশনা

1. নমুনা access log তৈরি করুন:

```
mkdir -p ~/labs/lab10 && cd ~/labs/lab10
cat > access.log <<'EOF'
192.168.1.10 - - [08/Nov/2026:10:00:01 +0600] "GET /index.html HTTP/1.1" 200 512
192.168.1.20 - - [08/Nov/2026:10:00:02 +0600] "GET /api/users HTTP/1.1" 200 1024
192.168.1.10 - - [08/Nov/2026:10:00:03 +0600] "GET /index.html HTTP/1.1" 200 512
192.168.1.30 - - [08/Nov/2026:10:00:04 +0600] "POST /api/login HTTP/1.1" 500 128
192.168.1.10 - - [08/Nov/2026:10:00:05 +0600] "GET /about.html HTTP/1.1" 404 256
192.168.1.20 - - [08/Nov/2026:10:00:06 +0600] "GET /api/users HTTP/1.1" 500 128
EOF
```

2. এক এক করে প্রশ্নের উত্তর বের করুন:

```
wc -l access.log
grep -c ' 500 ' access.log
grep ' 404 ' access.log
awk '{print $1}' access.log | sort | uniq -c | sort -nr
awk '{print $7}' access.log | sort | uniq -c | sort -nr
awk '$9 >= 400 {print $1, $7, $9}' access.log
```

3. একটি সারাংশ রিপোর্ট স্ক্রিপ্ট লিখুন:

```
cat > analyse.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
log_file="${1:-access.log}"
if [[ ! -f "$log_file" ]]; then
  printf 'Error: %s not found.\n' "$log_file" >&2
  exit 1
fi
printf 'Total requests: %s\n' "$(wc -l < "$log_file")"
printf 'Errors (4xx/5xx): %s\n' "$(awk '$9 >= 400' "$log_file" | wc -l)"
printf 'Top IPs:\n'
awk '{print $1}' "$log_file" | sort | uniq -c | sort -nr | head -n 5
printf 'Top URLs:\n'
awk '{print $7}' "$log_file" | sort | uniq -c | sort -nr | head -n 5
EOF
chmod +x analyse.sh
./analyse.sh access.log
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`awk '{print $1}'` প্রতিটি লাইনকে space দিয়ে ভেঙে প্রথম ক্ষেত্রটি (IP ঠিকানা) নেয়; `$7` হলো সপ্তম ক্ষেত্র — URL; `$9` status code। `sort | uniq -c` জোড়াটি গণনার মানক পদ্ধতি: `sort` একই মানগুলোকে পাশাপাশি সাজায়, `uniq -c` প্রতিটি স্বতন্ত্র মান কতবার এসেছে গোনে। `sort -nr` সংখ্যা অনুযায়ী বড় থেকে ছোট সাজায়। `awk '$9 >= 400'` একটি শর্ত — শুধু যে লাইনের status 400 বা বেশি, সেগুলোই ছাপে। `grep -c ' 500 '`-এ 500-র দুই পাশে space দেওয়া হয়েছে, যাতে 5001 বা 1500 ভুল করে না মেলে।

প্রত্যাশিত ফলাফল (illustrative)

```
Total requests: 6
Errors (4xx/5xx): 3
Top IPs:
      3 192.168.1.10
      2 192.168.1.20
      1 192.168.1.30
Top URLs:
      2 /index.html
      2 /api/users
      1 /about.html
      1 /api/login
```

যাচাই

- `grep -c ' 500 '` দেবে 2; `grep -c ' 404 '` দেবে 1।
- হাতে গুনে মোট 6 লাইন ও 3টি error মিলিয়ে দেখুন।
- আরেকটি লাইন log-এ যোগ করে আবার চালালে সংখ্যাগুলো বদলাবে।

> **TROUBLESHOOTING:** `awk`-এর ক্ষেত্র নম্বর ভুল হলে ফল অদ্ভুত আসবে — `head -n 1 access.log` দেখে ক্ষেত্রগুলো গুনুন। Log format আলাদা হলে (যেমন JSON log) এই ক্ষেত্র-ভিত্তিক পদ্ধতি চলবে না; তখন Python (Lab 15–16) ব্যবহার করুন।

> **WARNING:** Log-এ ব্যবহারকারীর ব্যক্তিগত তথ্য (IP, username) থাকতে পারে। রিপোর্ট শেয়ার করার আগে গোপনীয় তথ্য সরিয়ে নিন, আর log ফাইল অন্যের সঙ্গে শেয়ার করবেন না।

Cleanup: `cd ~ && rm -rf ~/labs/lab10`।

Lab 11 — Exit Code Handling

বাস্তব দৃশ্য: একটি nightly backup স্ক্রিপ্ট নীরবে ব্যর্থ হচ্ছে — ফাইল কপি না হলেও cron-এর log-এ কোনো error দেখা যায় না। কারণ স্ক্রিপ্টটি কোনো কমান্ডের exit code পরীক্ষা করে না। এবার প্রতিটি ধাপের exit code ধরে সঠিকভাবে ব্যর্থতা জানাবেন।

উদ্দেশ্য: `$?`, `exit`, `set -e`, `set -o pipefail` ও শর্তাধীন execution বোঝা ও ব্যবহার করা।

পূর্বজ্ঞান: Chapter 12, Lab 07–09। **পরিবেশ:** `~/labs/lab11`।

ধাপে ধাপে নির্দেশনা

1. Exit code-র আচরণ হাতে পরীক্ষা করুন:

```
mkdir -p ~/labs/lab11 && cd ~/labs/lab11
ls existing-file 2>/dev/null; echo "ls missing exit: $?"
ls /nonexistent; echo "ls missing exit: $?"
true; echo "true exit: $?"
false; echo "false exit: $?"
grep -q "hello" /etc/hostname; echo "grep exit: $?"
```

2. Pipeline-এ pipefail-এর পার্থক্য দেখুন:

```
set -o pipefail
grep "zzz-no-match" /etc/hostname | wc -l; echo "pipefail exit: $?"
set +o pipefail
grep "zzz-no-match" /etc/hostname | wc -l; echo "no pipefail exit: $?"
```

3. Exit code ধরে এমন একটি স্ক্রিপ্ট লিখুন:

```
cat > safe_backup.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
source_file="${1:-}"
dest_dir="${2:-$HOME/labs/lab11/backup}"
if [[ -z "$source_file" ]]; then
  printf 'Usage: safe_backup.sh <source> [dest-dir]\n' >&2
  exit 2
fi
if [[ ! -f "$source_file" ]]; then
  printf 'Error: source file not found: %s\n' "$source_file" >&2
  exit 1
fi
mkdir -p -- "$dest_dir"
if cp -- "$source_file" "$dest_dir/"; then
  printf 'Backup OK: %s\n' "$source_file"
  exit 0
else
  printf 'Error: copy failed for %s\n' "$source_file" >&2
  exit 1
fi
EOF
chmod +x safe_backup.sh
printf 'data\n' > data.txt
./safe_backup.sh data.txt; echo "exit: $?"
./safe_backup.sh missing.txt; echo "exit: $?"
./safe_backup.sh; echo "exit: $?"
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

Unix-এ প্রতিটি কমান্ড শেষ হওয়ার সময় একটি সংখ্যা ফেরত দেয় — `0` মানে সফল, অন্য যেকোনো সংখ্যা মানে ব্যর্থ। `$?` সর্বশেষ কমান্ডের exit code ধারণ করে, তবে পরের যেকোনো কমান্ড (এমনকি `echo`-ও) সেটি বদলে দেয় — তাই exit code পরীক্ষা করতে হলে ঠিক পরের লাইনেই করতে হয়। `set -e` কোনো কমান্ড ব্যর্থ হলে স্ক্রিপ্টকে তৎক্ষণাৎ থামিয়ে দেয়; `set -o pipefail` pipeline-এর যেকোনো অংশ ব্যর্থ হলে পুরো pipeline-কে ব্যর্থ ধরে — pipefail ছাড়া pipeline-এর

exit code শুধু শেষ কমান্ডের (`wc -l`) হতো, যা সবসময় সফল। এই lab-এর স্ক্রিপ্টে ভিন্ন ভিন্ন ভুলের জন্য ভিন্ন exit code ব্যবহার করা হয়েছে: 1 = সাধারণ ভুল, 2 = ভুল ব্যবহার।

প্রত্যাশিত ফলাফল (illustrative)

```
Backup OK: data.txt
exit: 0
Error: source file not found: missing.txt
exit: 1
Usage: safe_backup.sh <source> [dest-dir>
exit: 2
```

যাচাই

- তিনটি চালনার exit code যথাক্রমে 0, 1 ও 2 হবে।
- `ls backup`-এ `data.txt` থাকবে।
- `./safe_backup.sh data.txt && echo SUCCESS || echo FAILED` দেবে `SUCCESS`।

> **TROUBLESHOOTING:** `set -e` থাকা সত্ত্বেও `if`-এর ভেতরের ব্যর্থ কমান্ড স্ক্রিপ্ট থামায় না — এটি ইচ্ছাকৃত, কারণ `if` নিজেই সেই ব্যর্থতা সামলাচ্ছে। `$?` ভুল মান দিলে দেখুন তার আগে কোনো `echo` বা অন্য কমান্ড চলেছে কি না।

> **PRO TIP:** Cron job-এর সাফল্য-ব্যর্থতা exit code দিয়েই বোঝা যায়। Lab 18-এ এই স্ক্রিপ্টের exit code-ই cron log-এ ধরা হবে।

Cleanup: `cd ~ && rm -rf ~/labs/lab11`।

Lab 12 — Bash Debugging (set -x, ShellCheck)

বাস্তব দৃশ্য: একটি deploy স্ক্রিপ্ট ভুল ডিরেক্টরিতে ফাইল কপি করছে, কিন্তু কোথায় ভুল তা বোঝা যাচ্ছে না। স্ক্রিপ্টটি না বদলে তার ভেতরের প্রতিটি ধাপ দেখে ভুলটি খুঁজে বের করবেন — `set -x` trace ও ShellCheck দিয়ে।

উদ্দেশ্য: `bash -x`, `set -x`, `bash -n` ও ShellCheck ব্যবহার করে bug খুঁজে ঠিক করা।

পূর্বজ্ঞান: Chapter 12, Lab 07–11। **পরিবেশ:** `~/labs/lab12`।

ধাপে ধাপে নির্দেশনা

1. ইচ্ছাকৃতভাবে ভুলসহ একটি স্ক্রিপ্ট তৈরি করুন:

```
mkdir -p ~/labs/lab12 && cd ~/labs/lab12
cat > deploy.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
app_dir=/tmp/myapp
target_dir=$HOME/labs/lab12/deploy
mkdir -p $target_dir
cp $app_dir/*.txt $target_dir/
echo "Deployed to $target_dir"
```

```
EOF
mkdir -p /tmp/myapp
printf 'hello\n' > "/tmp/myapp/my file.txt"
chmod +x deploy.sh
```

2. ShellCheck চালান (না থাকলে install করুন):

```
sudo apt update && sudo apt install -y shellcheck
shellcheck deploy.sh
```

3. Trace mode-এ চালিয়ে ভুলটি দেখুন:

```
bash -x ./deploy.sh; echo "exit: $?"
```

4. স্ক্রিপ্টটি ঠিক করুন — সব variable quote করুন:

```
cat > deploy.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
app_dir="/tmp/myapp"
target_dir="$HOME/labs/lab12/deploy"
mkdir -p -- "$target_dir"
cp -- "$app_dir"/*.txt "$target_dir/"
printf 'Deployed to %s\n' "$target_dir"
EOF
shellcheck deploy.sh
bash -x ./deploy.sh
ls deploy
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

মূল ভুলটি ছিল quoting-এর অভাব: `cp $app_dir/*.txt`-এ `$app_dir` quote না থাকায় `my file.txt` ফাইলের নামের space-এ ভেঙে দুটি আলাদা argument হয়ে যায়, আর `cp` ব্যর্থ হয়। ShellCheck এই ভুলটি চালানোর আগেই ধরে — SC2086 সতর্কতা (double quote to prevent globbing and word splitting)। `bash -x` প্রতিটি কমান্ড চালানোর আগে `+` চিহ্নসহ দেখায়, variable-গুলো কী মান নিয়ে expand হয়েছে তাও দেখা যায় — এতে ভুলটি চোখে পড়ে। `bash -n` শুধু syntax পরীক্ষা করে, চালায় না — কোনো স্ক্রিপ্ট চালানোর আগে এটি একটি দ্রুত নিরাপত্তা পরীক্ষা।

প্রত্যাশিত ফলাফল (illustrative)

```
+ mkdir -p -- /home/nahid/labs/lab12/deploy
+ cp -- '/tmp/myapp/my file.txt' /home/nahid/labs/lab12/deploy/
+ printf 'Deployed to %s\n' /home/nahid/labs/lab12/deploy
Deployed to /home/nahid/labs/lab12/deploy
```

ঠিক করার পর `shellcheck deploy.sh` কোনো সতর্কতা দেবে না।

যাচাই

- `ls deploy`-এ `my file.txt` থাকবে।
- ShellCheck clean (কোনো আউটপুট নেই) হবে।
- `bash -n deploy.sh` কোনো error দেবে না।

TROUBLESHOOTING: Trace আউটপুট খুব বড় হলে শুধু সন্দেহজনক অংশের চারপাশে `set -x` ও `set +x` বসান। `set -x` trace-এ গোপন মান (পাসওয়ার্ড, টোকেন) দেখা যেতে পারে — debug করার সময় গোপন মানযুক্ত স্ক্রিপ্টের trace কাউকে পাঠাবেন না।

WARNING: `bash -x` চালানোর আগে স্ক্রিপ্টটি পড়ে নিন — অপরিচিত স্ক্রিপ্ট trace mode-এ চালালেও তা পুরোপুরি চলবে, শুধু দেখাবে না। অপরিচিত স্ক্রিপ্ট প্রথমে `bash -n` ও ShellCheck দিয়ে পরীক্ষা করুন।

Cleanup: `cd ~ && rm -rf ~/labs/lab12 /tmp/myapp`।

Lab 13 — Python File Automation

বাস্তব দৃশ্য: TechNova-র একটি ডিরেক্টরিতে প্রতিদিন শত শত `.log` ফাইল জমে। Python দিয়ে সেগুলোকে তারিখ অনুযায়ী সাজিয়ে archive করা, পুরনো ফাইল মুছে ফেলা ও একটি রিপোর্ট তৈরি করবেন — Bash-এর চেয়ে জটিল যুক্তি Python-এ লেখা সহজ।

উদ্দেশ্য: `pathlib`, `shutil`, তারিখ গণনা ও নিরাপদ ফাইল অপারেশন ব্যবহার করা।

পূর্বজ্ঞান: Chapter 14–15। **পরিবেশ:** `~/labs/lab13`, Python 3.12, virtual environment।

ধাপে ধাপে নির্দেশনা

1. Virtual environment তৈরি করুন ও পরীক্ষার ফাইল বানান:

```
mkdir -p ~/labs/lab13 && cd ~/labs/lab13
python3 -m venv .venv
source .venv/bin/activate
python3 --version
mkdir -p inbox archive
touch inbox/app-2026-11-01.log inbox/app-2026-11-08.log inbox/report.txt
python3 -c "import os; os.utime('inbox/app-2026-11-01.log', (0, 0))" 2>/dev/null || true
touch -d "10 days ago" inbox/app-2026-11-01.log
```

2. Automation স্ক্রিপ্টটি লিখুন:

```
cat > organize.py <<'EOF'
import shutil
import sys
from datetime import datetime, timedelta
from pathlib import Path

def organize(inbox: Path, archive: Path, days: int = 7) -> dict:
    cutoff = datetime.now() - timedelta(days=days)
    result = {"archived": [], "kept": [], "skipped": []}
    if not inbox.is_dir():
        raise SystemExit(f"Error: inbox not found: {inbox}")
    archive.mkdir(parents=True, exist_ok=True)
    for entry in sorted(inbox.iterdir()):
        if entry.is_dir():
            result["skipped"].append(entry.name)
            continue
        if entry.suffix != ".log":
```

```
            result["skipped"].append(entry.name)
            continue
        modified = datetime.fromtimestamp(entry.stat().st_mtime)
        if modified < cutoff:
            target = archive / entry.name
            shutil.move(str(entry), str(target))
            result["archived"].append(entry.name)
        else:
            result["kept"].append(entry.name)
    return result

def main() -> int:
    inbox = Path("inbox")
    archive = Path("archive")
    result = organize(inbox, archive, days=7)
    print(f"Archived: {len(result['archived'])}")
    for name in result["archived"]:
        print(f"  archived: {name}")
    print(f"Kept: {len(result['kept'])}")
    for name in result["kept"]:
        print(f"  kept: {name}")
    print(f"Skipped: {len(result['skipped'])}")
    return 0

if __name__ == "__main__":
    sys.exit(main())
EOF
python3 organize.py
ls inbox archive
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`pathlib.Path` ফাইলের পথকে একটি object হিসেবে ধরে — `entry.is_dir()`, `entry.suffix`, `entry.stat().st_mtime` সবই এর method। String দিয়ে পথ জোড়া লাগানোর চেয়ে এটি নিরাপদ, কারণ operating system-ভেদে separator-এর পার্থক্য Path নিজে সামলায়। `datetime.now() - timedelta(days=7)` ঠিক ৭ দিন আগের সময় বের করে; ফাইলের `st_mtime` সেই সময়ের চেয়ে পুরনো হলে সেটি archive-এর যোগ্য। `shutil.move` ফাইল সরায় — একই filesystem-এ rename, আলাদা filesystem-এ copy-then-delete। `if __name__ == "__main__"` গার্ডটি নিশ্চিত করে এই ফাইলটি import করলে `main()` চলবে না, শুধু সরাসরি চালালেই চলবে — পরীক্ষার জন্য এটি অপরিহার্য।

প্রত্যাশিত ফলাফল (illustrative)

```
Archived: 1
  archived: app-2026-11-01.log
Kept: 1
  kept: app-2026-11-08.log
Skipped: 1
```

যাচাই

- `ls archive`-এ শুধু পুরনো log ফাইলটি থাকবে; `report.txt` inbox-এই থাকবে।
- আবার চালালে `Archived: 0` হবে — idempotent।
- `python3 -m py_compile organize.py` কোনো error দেবে না।

TROUBLESHOOTING: `PermissionError` এলে ফাইলটি অন্য প্রোগ্রাম খুলে রেখেছে কি না দেখুন। `FileNotFoundError` এলে বর্তমান ডিরেক্টরি ঠিক আছে কি না (`pwd`) দেখুন — স্ক্রিপ্টটি `~/labs/lab13` থেকে চালাতে হয়।

WARNING: ফাইল মোছা বা সরানোর স্ক্রিপ্টে সবসময় প্রথমে dry-run (শুধু কী করত তা ছাপা) যোগ করুন; Lab 22-এ এই প্যাটার্নটি পূর্ণাঙ্গভাবে শেখা হবে। আসল ডেটা দিয়ে পরীক্ষা করবেন না।

Cleanup: `deactivate; cd ~ && rm -rf ~/labs/lab13`।

Lab 14 — Python Subprocess Automation

বাস্তব দৃশ্য: একটি Python মনিটরিং টুল থেকে Linux কমান্ড (`df`, `free`, `systemctl`) চালিয়ে ফলাফল সংগ্রহ করতে হবে — কিন্তু shell injection-এর ঝুঁকি ছাড়া, timeout-সহ, stdout ও stderr আলাদাভাবে ধরে।

উদ্দেশ্য: `subprocess.run` নিরাপদভাবে ব্যবহার করা; argument list, `capture_output`, `timeout`, exit code ও `text=True` বোঝা।

পূর্বজ্ঞান: Chapter 16, Lab 13। **পরিবেশ:** `~/labs/lab14`, Python 3.12।

ধাপে ধাপে নির্দেশনা

1. পরিবেশ তৈরি করুন:

```
mkdir -p ~/labs/lab14 && cd ~/labs/lab14
python3 -m venv .venv
source .venv/bin/activate
```

2. স্ক্রিপ্টটি লিখুন:

```
cat > run_cmd.py <<'EOF'
import subprocess
import sys

def run_command(argv, timeout=10):
    try:
        result = subprocess.run(
            argv,
            capture_output=True,
            text=True,
            timeout=timeout,
            check=False,
        )
    except subprocess.TimeoutExpired:
        print(f"Error: command timed out after {timeout}s: {argv[0]}")
        return 1
    except FileNotFoundError:
        print(f"Error: command not found: {argv[0]}")
        return 1
    print(f"Command: {' '.join(argv)}")
    print(f"Exit code: {result.returncode}")
    if result.stdout:
```

```
        print("--- stdout ---")
        print(result.stdout.rstrip())
    if result.stderr:
        print("--- stderr ---")
        print(result.stderr.rstrip())
    return result.returncode

def main():
    code = 0
    code = run_command(["df", "-h", "/"]) or code
    code = run_command(["free", "-h"]) or code
    code = run_command(["systemctl", "is-active", "ssh"]) or code
    code = run_command(["ls", "/nonexistent-path"]) or code
    code = run_command(["sleep", "30"], timeout=2) or code
    return code

if __name__ == "__main__":
    sys.exit(main())
EOF
python3 run_cmd.py; echo "script exit: $?"
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

সবচেয়ে গুরুত্বপূর্ণ সিদ্ধান্ত: কমান্ডটি একটি **list** হিসেবে দেওয়া হয়েছে (`["df", "-h", "/"]`), কোনো shell string নয়, আর `shell=True` ব্যবহার করা হয়নি। এতে ব্যবহারকারীর দেওয়া মান (যেমন ফাইলের নাম) কখনো shell কমান্ড হিসেবে ব্যাখ্যা হবে না — shell injection অসম্ভব। `capture_output=True` stdout ও stderr দুটোই ধরে; `text=True` সেগুলোকে bytes-এর বদলে str হিসেবে দেয়। `timeout=10` কমান্ড আটকে গেলে ১০ সেকেন্ড পর `TimeoutExpired` তোলে — `sleep 30` তাই ২ সেকেন্ডে থেমে যায়। `check=False` মানে ব্যর্থ কমান্ড exception তুলবে না; আমরা নিজেরা `returncode` পরীক্ষা করব — এটি automation-এ সঠিক প্যাটার্ন, কারণ ব্যর্থতাও একটি তথ্য।

প্রত্যাশিত ফলাফল (illustrative)

```
Command: df -h /
Exit code: 0
--- stdout ---
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   21G   27G  44% /
Command: ls /nonexistent-path
Exit code: 2
--- stderr ---
ls: cannot access '/nonexistent-path': No such file or directory
Command: sleep 30
Error: command timed out after 2s: sleep
```

যাচাই

- `df` ও `free` -এর আউটপুট আপনার সিস্টেমের আসল মানের সঙ্গে মিলবে।
- `ls` -এর exit code 2 হবে, কিন্তু স্ক্রিপ্ট থামবে না — সব কমান্ড চলবে।
- স্ক্রিপ্টটি `sleep 30` -এ আটকে থাকবে না; মোট সময় ~২ সেকেন্ডের বেশি হবে না।

> **TROUBLESHOOTING:** `FileNotFoundError` এলে কমান্ডটি install আছে কি না দেখুন (`which df`)। `TimeoutExpired` ধরা না হলে traceback আসবে — `try/except` ঠিক জায়গায় আছে কি না দেখুন।

> **WARNING:** `shell=True` ব্যবহার করবেন না, বিশেষ করে ব্যবহারকারীর input কমান্ডে জোড়া লাগিয়ে। একটি ক্ষতিকর ফাইলের নাম যেমন `file.txt; rm -rf ~` shell-এ চালালে বিপর্যয় ঘটতে পারে; argument list-এ তা শুধু একটি ফাইলের নাম।

Cleanup: `deactivate; cd ~ && rm -rf ~/labs/lab14`।

Lab 15 — API Collection (Public API থেকে JSON সংগ্রহ)

বাস্তব দৃশ্য: TechNova-র একটি ড্যাশবোর্ডে GitHub-এর একটি public repository-র তথ্য (তারকা সংখ্যা, fork, সর্বশেষ আপডেট) দেখাতে হবে। GitHub API থেকে JSON সংগ্রহ করে প্রয়োজনীয় ক্ষেত্রগুলো বের করবেন — কোনো গোপন token ছাড়াই, কারণ public তথ্যের জন্য token লাগে না।

উদ্দেশ্য: `requests` দিয়ে HTTP GET করা, status code পরীক্ষা করা, JSON parse করা ও timeout/retry সামলানো।

পূর্বজ্ঞান: Chapter 17, Lab 13। **পরিবেশ:** `~/labs/lab15`, Python 3.12, ইন্টারনেট।

ধাপে ধাপে নির্দেশনা

1. পরিবেশ তৈরি করুন:

```
mkdir -p ~/labs/lab15 && cd ~/labs/lab15
python3 -m venv .venv
source .venv/bin/activate
pip install requests
```

2. স্ক্রিপ্টটি লিখুন:

```
cat > fetch_repo.py <<'EOF'
import sys
import requests

def fetch_repo(owner, repo, timeout=10):
    url = f"https://api.github.com/repos/{owner}/{repo}"
    headers = {
        "Accept": "application/vnd.github+json",
        "User-Agent": "technova-lab/1.0",
    }
    try:
        response = requests.get(url, headers=headers, timeout=timeout)
    except requests.exceptions.Timeout:
        print("Error: request timed out.", file=sys.stderr)
        return None
    except requests.exceptions.RequestException as exc:
        print(f"Error: network problem: {exc}", file=sys.stderr)
        return None
    if response.status_code == 404:
        print(f"Error: repository not found: {owner}/{repo}", file=sys.stderr)
        return None
```

```
    if response.status_code == 403:
        print("Error: rate limited. Wait and try again later.", file=sys.stderr)
        return None
    if response.status_code != 200:
        print(f"Error: unexpected status {response.status_code}", file=sys.stderr)
        return None
    return response.json()

def main():
    owner = sys.argv[1] if len(sys.argv) > 1 else "python"
    repo = sys.argv[2] if len(sys.argv) > 2 else "cpython"
    data = fetch_repo(owner, repo)
    if data is None:
        return 1
    print(f"Repository: {data.get('full_name')}")
    print(f"Description: {data.get('description')}")
    print(f"Stars: {data.get('stargazers_count')}")
    print(f"Forks: {data.get('forks_count')}")
    print(f"Open issues: {data.get('open_issues_count')}")
    print(f"Language: {data.get('language')}")
    print(f"Updated at: {data.get('updated_at')}")
    return 0

if __name__ == "__main__":
    sys.exit(main())
EOF
python3 fetch_repo.py python cpython
python3 fetch_repo.py nosuchowner123 nosuchrepo123; echo "exit: $?"
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`requests.get(url, timeout=10)`-এ timeout দেওয়া অপরিহার্য — timeout ছাড়া সার্ভার সাড়া না দিলে প্রোগ্রাম অনির্দিষ্টকাল আটকে থাকতে পারে। Status code পরীক্ষা করা হয়েছে JSON parse করার **আগে** — 404 বা 403 উত্তরে JSON কাঠামো আলাদা হতে পারে, সরাসরি parse করলে বিভ্রান্তিকর error আসত। `data.get('description')` ব্যবহার করা হয়েছে `data['description']`-এর বদলে, যাতে কোনো ক্ষেত্র না থাকলে `KeyError` না উঠে `None` আসে। `User-Agent` header দেওয়া হয়েছে কারণ অনেক API অজ্ঞাত client-কে সীমিত করে। এই lab-এ কোনো token ব্যবহার করা হয়নি — public তথ্যের জন্য প্রয়োজন নেই, আর token থাকলে তা কখনো কোডে লিখবেন না, পরিবেশ চলক থেকে পড়বেন (Chapter 18)।

প্রত্যাশিত ফলাফল (illustrative)

```
Repository: python/cpython
Description: The Python programming language
Stars: 68000
Forks: 32000
Open issues: 8500
Language: Python
Updated at: 2026-11-08T09:00:00Z
```

সংখ্যাগুলো আপনার চালানোর সময়ের আসল মান হবে; ভুল repository-র ক্ষেত্রে `Error: repository not found` ও exit code 1 আসবে।

যাচাই

- ব্রাউজারে `https://api.github.com/repos/python/cpython` খুলে তারকা সংখ্যা মিলিয়ে দেখুন।
- ইন্টারনেট বন্ধ করে চালালে network error বার্তা আসবে, traceback নয়।
- `python3 fetch_repo.py torvalds linux` অন্য repository-র তথ্য দেবে।

> **TROUBLESHOOTING:** `ModuleNotFoundError: requests` এলে virtual environment activate আছে কি না দেখুন। `403 rate limited` এলে এক ঘণ্টা অপেক্ষা করুন — unauthenticated অনুরোধে GitHub প্রতি ঘণ্টায় ৬০টি অনুরোধ দেয়। `verify=False` দিয়ে TLS যাচাই বন্ধ করবেন না — এটি নিরাপত্তা ঝুঁকি; certificate সমস্যা হলে সিস্টেমের certificate হালনাগাদ করুন।

> **WARNING:** API token কখনো source code-এ, Git commit-এ বা স্ক্রিনশটে রাখবেন না। Token লাগলে পরিবেশ চলক (`os.environ.get("GITHUB_TOKEN")`) থেকে পড়ুন।

Cleanup: `deactivate; cd ~ && rm -rf ~/labs/lab15` ।

Lab 16 — JSON ও YAML Processing

বাস্তব দৃশ্য: TechNova-র সার্ভার তালিকা একটি YAML ফাইলে আছে, আর একটি API JSON দেয়। আপনাকে YAML পড়ে সার্ভারগুলোর তালিকা বের করতে হবে, JSON-এ রূপান্তর করতে হবে, ভুল কাঠামো ধরে ফেলতে হবে এবং ফলাফল একটি JSON রিপোর্টে লিখতে হবে।

উদ্দেশ্য: `json` ও `yaml` মডিউল দিয়ে পড়া-লেখা, কাঠামো যাচাই (validation) ও নিরাপদ YAML লোড করা।

পূর্বজ্ঞান: Chapter 18, Lab 13। **পরিবেশ:** `~/labs/lab16` , Python 3.12।

ধাপে ধাপে নির্দেশনা

1. পরিবেশ ও নমুনা ফাইল তৈরি করুন:

```
mkdir -p ~/labs/lab16 && cd ~/labs/lab16
python3 -m venv .venv
source .venv/bin/activate
pip install pyyaml
cat > servers.yaml <<'EOF'
environment: staging
servers:
  - name: web-01
    host: 192.168.1.10
    role: web
    port: 22
  - name: db-01
    host: 192.168.1.11
    role: database
    port: 22
EOF
cat > sample.json <<'EOF'
{"project": "technova", "servers": 2, "healthy": true}
EOF
```

2. প্রসেসিং স্ক্রিপ্ট লিখুন:

```
cat > process_config.py <<'EOF'
import json
import sys
from pathlib import Path
import yaml

def load_yaml(path):
    with open(path, "r", encoding="utf-8") as handle:
        data = yaml.safe_load(handle)
    if not isinstance(data, dict):
        raise ValueError("YAML root must be a mapping.")
    servers = data.get("servers")
    if not isinstance(servers, list) or not servers:
        raise ValueError("YAML must contain a non-empty 'servers' list.")
    for server in servers:
        for field in ("name", "host", "role"):
            if field not in server:
                raise ValueError(f"Server entry missing field: {field}")
    return data

def main() -> int:
    try:
        config = load_yaml(Path("servers.yaml"))
    except FileNotFoundError:
        print("Error: servers.yaml not found.", file=sys.stderr)
        return 1
    except (yaml.YAMLError, ValueError) as exc:
        print(f"Error: invalid configuration: {exc}", file=sys.stderr)
        return 1
    with open("sample.json", "r", encoding="utf-8") as handle:
        summary = json.load(handle)
    report = {
        "environment": config.get("environment", "unknown"),
        "project": summary.get("project"),
        "server_count": len(config["servers"]),
        "servers": [
            {"name": s["name"], "host": s["host"], "role": s["role"]}
            for s in config["servers"]
        ],
    }
    with open("report.json", "w", encoding="utf-8") as handle:
        json.dump(report, handle, indent=2)
    print(f"Environment: {report['environment']}")
    print(f"Servers: {report['server_count']}")
    for server in report["servers"]:
        print(f"  {server['name']} ({server['role']}) -> {server['host']}")
    print("Wrote report.json")
    return 0

if __name__ == "__main__":
    sys.exit(main())
EOF
python3 process_config.py
cat report.json
```

3. ভুল YAML দিয়ে validation পরীক্ষা করুন:

```
printf 'servers: []\n' > bad.yaml
python3 -c "import process_config;
process_config.load_yaml(__import__('pathlib').Path('bad.yaml'))" 2>&1 | head -n 3; echo
"exit: $?"
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`yaml.safe_load` ব্যবহার করা হয়েছে, `yaml.load` নয় — `safe_load` শুধু সাধারণ ডেটা টাইপ পড়ে, YAML-এর ভেতরে লুকানো Python object তৈরি হতে দেয় না; অনিরাপদ `load` দিয়ে ক্ষতিকর YAML কোড চালানো সম্ভব। Validation-এর ধাপটি গুরুত্বপূর্ণ: ফাইলটি YAML হিসেবে বৈধ হলেই যথেষ্ট নয় — `servers` তালিকা আছে কি না, প্রতিটি সার্ভারে `name`, `host`, `role` আছে কি না, তা আলাদাভাবে পরীক্ষা করা হয়েছে। এতে ভুল configuration দেরিতে (production-এ গিয়ে) নয়, শুরুতেই ধরা পড়ে। `json.dump(report, handle, indent=2)` মানুষের পড়ার মতো সুন্দর JSON লেখে; `indent` ছাড়া সব এক লাইনে আসত।

প্রত্যাশিত ফলাফল (illustrative)

```
Environment: staging
Servers: 2
  web-01 (web) -> 192.168.1.10
  db-01 (database) -> 192.168.1.11
Wrote report.json
```

যাচাই

- `cat report.json` বৈধ JSON হবে — `python3 -m json.tool report.json` কোনো error দেবে না।
- `bad.yaml` পরীক্ষায় `ValueError` বার্তা আসবে।
- YAML-এ একটি সার্ভারের `host` মুছে চালালে validation error আসবে।

TROUBLESHOOTING: `yaml.scanner.ScannerError` এলে indentation দেখুন — YAML-এ tab ব্যবহার করা যায় না, শুধু space। `ModuleNotFoundError: yaml` এলে `pip install pyyaml` হয়েছে কি না ও venv activate আছে কি না দেখুন।

WARNING: YAML ফাইলে কখনো আসল পাসওয়ার্ড বা API key লিখবেন না। গোপন মান পরিবেশ চলক বা secret manager-এ রাখুন, আর configuration ফাইল Git-এ commit করার আগে `.gitignore` পরীক্ষা করুন।

Cleanup: `deactivate; cd ~ && rm -rf ~/labs/lab16`।

Lab 17 — Structured Logging

বাস্তব দৃশ্য: একটি backup টুলের log পড়ে বোঝা যাচ্ছে না কোন backup কখন ব্যর্থ হয়েছে, আর কেন। `print`-এর বদলে Python-এর `logging` মডিউল দিয়ে timestamp, level ও প্রসঙ্গসহ structured log লিখবেন — ফাইলে rotating log সহ।

উদ্দেশ্য: Logger তৈরি, level ব্যবহার, formatter, file handler ও `RotatingFileHandler` শেখা।

পূর্বজ্ঞান: Chapter 19, Lab 13। **পরিবেশ:** `~/labs/lab17`, Python 3.12।

ধাপে ধাপে নির্দেশনা

1. পরিবেশ তৈরি করুন:

```
mkdir -p ~/labs/lab17 && cd ~/labs/lab17
python3 -m venv .venv
source .venv/bin/activate
```

2. Logging-সহ স্ক্রিপ্ট লিখুন:

```
cat > backup_tool.py <<'EOF'
import logging
import sys
from logging.handlers import RotatingFileHandler
from pathlib import Path

def build_logger():
    logger = logging.getLogger("technova.backup")
    logger.setLevel(logging.DEBUG)
    formatter = logging.Formatter(
        "%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )
    console = logging.StreamHandler(sys.stdout)
    console.setLevel(logging.INFO)
    console.setFormatter(formatter)
    log_file = RotatingFileHandler(
        "backup.log", maxBytes=10_000, backupCount=3, encoding="utf-8"
    )
    log_file.setLevel(logging.DEBUG)
    log_file.setFormatter(formatter)
    logger.addHandler(console)
    logger.addHandler(log_file)
    return logger

def backup_file(logger, source, dest_dir):
    logger.info("Starting backup of %s", source)
    source_path = Path(source)
    if not source_path.is_file():
        logger.error("Source file not found: %s", source)
        return False
    try:
        dest = Path(dest_dir) / source_path.name
        dest.parent.mkdir(parents=True, exist_ok=True)
        data = source_path.read_bytes()
        logger.debug("Read %d bytes from %s", len(data), source)
        dest.write_bytes(data)
    except OSError as exc:
        logger.exception("Backup failed for %s: %s", source, exc)
        return False
    logger.info("Backup complete: %s -> %s", source, dest)
    return True

def main() -> int:
    logger = build_logger()
    logger.info("Backup tool started")
    ok = backup_file(logger, "data.txt", "backups")
    backup_file(logger, "missing.txt", "backups")
```

```
    logger.info("Backup tool finished")
    return 0 if ok else 1

if __name__ == "__main__":
    sys.exit(main())
EOF
printf 'important data\n' > data.txt
python3 backup_tool.py
cat backup.log
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

Logger-এর দুটি handler আছে: console handler শুধু INFO ও তার উপরের বার্তা দেখায়, আর file handler DEBUG পর্যন্ত সবকিছু লেখে — এতে টার্মিনালে সংক্ষিপ্ত সারাংশ আর ফাইলে বিস্তারিত তদন্তের তথ্য থাকে। Formatter-এ timestamp, level ও logger-এর নাম আছে — কোনো log লাইন দেখলেই বোঝা যায় কখন, কতটা গুরুতর, কোন অংশ থেকে এসেছে। `RotatingFileHandler(maxBytes=10_000, backupCount=3)` ফাইলটি ১০ KB ছাড়ালে পুরনো ফাইলকে `backup.log.1` করে নতুন ফাইল শুরু করে, সর্বোচ্চ ৩টি পুরনো ফাইল রাখে — disk ভরে যাওয়া ঠেকায়। `logger.exception` শুধু `except` ব্লকের ভেতরে ব্যবহার করা হয়; এটি বার্তার সঙ্গে পুরো traceback লেখে। আরেকটি সূক্ষ্ম বিষয়: `logger.info("Starting backup of %s", source)` — বার্তাটি format করা হয় শুধু তখনই যখন level অনুমতি দেয়; f-string দিয়ে আগেই format করলে অপ্রয়োজনীয় কাজ হতো।

প্রত্যাশিত ফলাফল (illustrative)

```
2026-11-08 10:00:00 | INFO     | technova.backup | Backup tool started
2026-11-08 10:00:00 | INFO     | technova.backup | Starting backup of data.txt
2026-11-08 10:00:00 | INFO     | technova.backup | Backup complete: data.txt -> backups/
data.txt
2026-11-08 10:00:00 | INFO     | technova.backup | Starting backup of missing.txt
2026-11-08 10:00:00 | ERROR    | technova.backup | Source file not found: missing.txt
```

`backup.log`-এ একই লাইনগুলোর সঙ্গে DEBUG লাইন (`Read 15 bytes`) যোগ থাকবে।

যাচাই

- `ls backups`-এ `data.txt` থাকবে।
- Console-এ DEBUG লাইন দেখা যাবে না, কিন্তু `grep DEBUG backup.log` দিলে পাওয়া যাবে।
- বড় ফাইল বারবার লিখে `backup.log.1` তৈরি হওয়া পরীক্ষা করতে পারেন।

TROUBLESHOOTING: একই বার্তা দুবার দেখা গেলে logger-এ handler দুবার যোগ হয়েছে — `build_logger()` একাধিকবার না ডাকার বিষয়টি দেখুন। Log ফাইল খালি থাকলে level দেখুন: root logger-এর level DEBUG না হলে DEBUG বার্তা আসবে না।

WARNING: Log-এ কখনো পাসওয়ার্ড, টোকেন বা ব্যক্তিগত তথ্য লিখবেন না। `logger.debug("token=%s", token)` ধরনের লাইন production-এ বিপজ্জনক।

Cleanup: `deactivate; cd ~ && rm -rf ~/labs/lab17`।

Lab 18 — Cron Jobs: নির্ধারিত স্বয়ংক্রিয় কাজ

বাস্তব দৃশ্য: TechNova-র সার্ভারে প্রতিদিন রাত ২টায় একটি health-check স্ক্রিপ্ট চলবে, আর প্রতি ঘণ্টায় একটি disk-check। মানুষ মনে করে চালাবে না — cron দিয়ে সময়মতো চালানোর ব্যবস্থা করবেন, log-সহ, আর একই job দুবার চলা ঠেকাবেন।

উদ্দেশ্য: Crontab লেখা, cron-এর পরিবেশের পার্থক্য বোঝা, `flock` দিয়ে duplicate execution ঠেকানো ও log করা।

পূর্বজ্ঞান: Chapter 20, Lab 11, Lab 17। **পরিবেশ:** Ubuntu 24.04 LTS, `~/labs/lab18`।

ধাপে ধাপে নির্দেশনা

1. Health-check স্ক্রিপ্ট লিখুন (সম্পূর্ণ পথ ব্যবহার করে):

```
mkdir -p ~/labs/lab18 && cd ~/labs/lab18
cat > healthcheck.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
LOG_FILE="$HOME/labs/lab18/health.log"
{
  printf '[%s] Health check started\n' "$(date '+%Y-%m-%d %H:%M:%S')"
  printf 'Disk: %s\n' "$(df -h / | tail -n 1)"
  printf 'Memory: %s\n' "$(free -h | awk 'NR==2 {print $3 "/" $2}')"
  printf 'Load: %s\n' "$(uptime)"
  printf '[%s] Health check finished\n' "$(date '+%Y-%m-%d %H:%M:%S')"
} >> "$LOG_FILE" 2>&1
EOF
chmod +x healthcheck.sh
./healthcheck.sh
cat health.log
```

2. Crontab দেখুন ও পরীক্ষার জন্য প্রতি মিনিটে চালানোর entry যোগ করুন:

```
crontab -l || true
crontab -e
```

এডিটরে এই লাইনটি যোগ করুন (আপনার আসল home পথ বসান):

```
* * * * * /usr/bin/flock -n /home/nahid/labs/lab18/health.lock /home/nahid/labs/lab18/
healthcheck.sh
```

3. দুই মিনিট অপেক্ষা করে log দেখুন:

```
sleep 130
cat health.log
crontab -l
```

4. পরীক্ষা শেষে entry-টি সরিয়ে ফেলুন:

```
crontab -e
```

পরীক্ষার লাইনটি মুছে সংরক্ষণ করুন। তারপর যাচাই করুন:

```
crontab -l
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

Cron entry-র পাঁচটি ক্ষেত্র: মিনিট, ঘণ্টা, মাসের দিন, মাস, সপ্তাহের দিন। `* * * * *` মানে প্রতি মিনিটে; দৈনিক রাত ২টায় চালাতে লিখতে হয় `0 2 * * *`। Cron-এর পরিবেশ আপনার interactive shell-এর মতো নয় — `PATH` খুব ছোট, home ডিরেক্টরি বা virtual environment সক্রিয় থাকে না। তাই স্ক্রিপ্টে `PATH` স্পষ্ট করে দেওয়া হয়েছে, আর সব পথ সম্পূর্ণ (absolute) লেখা হয়েছে; `~/` বা আপেক্ষিক পথ cron-এ নির্ভরযোগ্য নয়। `flock -n` একটি lock ফাইল ধরে: আগের চালনা এখনো চলতে থাকলে নতুন চালনা শুরু না হয়ে চুপচাপ থেমে যায় — দীর্ঘ job দুবার চলা ঠেকায়। আউটপুট `>> "$LOG_FILE" 2>&1` দিয়ে ফাইলে পাঠানো হয়েছে, নইলে cron প্রতিবার ইমেইল পাঠানোর চেষ্টা করত।

প্রত্যাশিত ফলাফল (illustrative)

```
[2026-11-08 10:02:00] Health check started
Disk: /dev/sda1  50G  21G  27G  44% /
Memory: 1.2Gi/3.8Gi
Load: 10:02:00 up 3 days, 1 user, load average: 0.10, 0.05, 0.01
[2026-11-08 10:02:00] Health check finished
```

দুই মিনিট পর log-এ অন্তত দুটি চালনার রেকর্ড থাকবে।

যাচাই

- `crontab -l` পরীক্ষার entry দেখাবে, আর মুছে ফেলার পর দেখাবে না।
- Log ফাইলে timestamp-গুলো এক মিনিটের ব্যবধানে থাকবে।
- স্ক্রিপ্টটি হাতে চালালেও একই ফরম্যাটে log লিখবে।

> **TROUBLESHOOTING:** Cron job চলছে না মনে হলে `grep CRON /var/log/syslog` দেখুন (অনুমতি লাগতে পারে)। সবচেয়ে সাধারণ কারণ: আপেক্ষিক পথ, ছোট PATH, বা স্ক্রিপ্টে executable permission না থাকা। `crontab -e`-তে syntax ভুল থাকলে cron পুরো ফাইলটি প্রত্যাখ্যান করতে পারে — সংরক্ষণের পর `crontab -l` দিয়ে যাচাই করুন।

> **WARNING:** পরীক্ষার `* * * * *` entry ভুলে রেখে যাবেন না — প্রতি মিনিটে চলা job disk ভরে ফেলতে পারে। Production-এ cron job-এর আগে সবসময় হাতে চালিয়ে পরীক্ষা করুন, আর lock ফাইল ব্যবহার করুন।

Cleanup: Crontab থেকে পরীক্ষার লাইন সরান; `cd ~ && rm -rf ~/labs/lab18`।

Lab 19 — SSH Remote Operations

বাস্তব দৃশ্য: TechNova-র দ্বিতীয় সার্ভারে (staging) একটি health-check চালাতে হবে, আর একটি configuration ফাইল সেখানে পাঠাতে হবে — সার্ভারে বসে নয়, নিজের মেশিন থেকে। পাসওয়ার্ড টাইপ না করে key-based SSH দিয়ে নিরাপদে করবেন।

উদ্দেশ্য: SSH key তৈরি, public key স্থাপন, remote command চালানো, `scp` ও `rsync` দিয়ে ফাইল পাঠানো।

পূর্বজ্ঞান: Chapter 21, Lab 06। **পরিবেশ:** দুটি Linux মেশিন বা একটি local VM (এই lab-এ `localhost`-এ SSH সার্ভার দিয়ে অনুশীলন করা যাবে); Ubuntu 24.04 LTS, OpenSSH।

ধাপে ধাপে নির্দেশনা

1. SSH সার্ভার ও key প্রস্তুত করুন:

```
sudo apt update && sudo apt install -y openssh-server openssh-client
sudo systemctl enable --now ssh
ssh-keygen -t ed25519 -C "nahid@example.com"
ls -l ~/.ssh/
```

2. নিজের public key নিজের authorized_keys-এ যোগ করুন (পরীক্ষার জন্য localhost):

```
mkdir -p ~/.ssh
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

3. প্রথম সংযোগে host key যাচাই করুন:

```
ssh -o StrictHostKeyChecking=accept-new localhost 'echo connected; hostname; uptime'
```

4. Remote command চালান ও ফাইল পাঠান:

```
ssh localhost 'df -h / | tail -n 1'
ssh localhost 'free -h | head -n 2'
printf 'timeout=45\n' > app.conf
scp app.conf localhost:/tmp/app.conf
ssh localhost 'cat /tmp/app.conf'
```

5. একটি ডিরেক্টরি rsync দিয়ে sync করুন:

```
mkdir -p myproject
printf 'v1\n' > myproject/app.txt
rsync -av --delete myproject/ localhost:/tmp/myproject/
ssh localhost 'ls -l /tmp/myproject/'
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

SSH key জোড়ায় private key আপনার মেশিনে থাকে, public key সার্ভারে যায় — সার্ভার একটি চ্যালেঞ্জ পাঠায় যা শুধু private key দিয়ে সমাধান করা যায়, তাই পাসওয়ার্ড নেটওয়ার্কে যায় না। `~/.ssh`-এর permission 700 ও `authorized_keys`-এর 600 হওয়া বাধ্যতামূলক — ঢিলা permission থাকলে SSH key-কে উপেক্ষা করে। প্রথম সংযোগে সার্ভারের host key দেখানো হয়; `StrictHostKeyChecking=accept-new` অজানা host-এর key প্রথমবার গ্রহণ করে সংরক্ষণ করে, কিন্তু পরে key বদলে গেলে সতর্ক করে — এটি `StrictHostKeyChecking=no`-এর চেয়ে নিরাপদ, কারণ পরেরটি man-in-the-middle আক্রমণ লুকিয়ে ফেলে। `rsync -av --delete` শুধু বদলে যাওয়া ফাইল পাঠায়, আর গন্তব্যের অতিরিক্ত ফাইল মুছে দেয় — বড় deploy-এ `scp`-এর চেয়ে দ্রুত।

প্রত্যাশিত ফলাফল (illustrative)

```
connected
ubuntu-server
 10:00:00 up 3 days, 1 user, load average: 0.10, 0.05, 0.01
timeout=45
sending incremental file list
app.txt
```

যাচাই

- `ssh localhost 'echo ok'` পাসওয়ার্ড না চেয়ে `ok` দেবে।
- `ssh localhost 'cat /tmp/app.conf'`-এ `timeout=45` দেখা যাবে।
- `rsync`-এর পর remote ডিরেক্টরিতে `app.txt` থাকবে।

> **TROUBLESHOOTING:** `Permission denied (publickey)` এলে `~/.ssh` ও `authorized_keys`-এর permission দেখুন, আর সার্ভারের `/var/log/auth.log`-এ কারণ খুঁজুন। `WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED` এলে সার্ভারটি সত্যিই নতুন করে install হয়েছে কি না নিশ্চিত না হয়ে `known_hosts` থেকে লাইন মুছবেন না।

> **WARNING:** Private key কখনো শেয়ার করবেন না, ইমেইলে পাঠাবেন না, Git-এ commit করবেন না। Key-তে passphrase দিন। সার্ভারে root login ও password authentication বন্ধ রাখা ভালো অভ্যাস — `/etc/ssh/sshd_config`-এ `PermitRootLogin no` ও `PasswordAuthentication no` যাচাই করুন।

Cleanup: `ssh localhost 'rm -rf /tmp/myproject /tmp/app.conf'`; পরীক্ষার key নিজের মেশিনে রাখলে তা শুধু পরীক্ষার জন্য — আসল সার্ভারে আলাদা key ব্যবহার করুন। `cd ~ && rm -rf ~/labs/lab19 2>/dev/null; true` (এই lab-এ lab19 ডিরেক্টরি তৈরি করা হয়নি, তাই কিছু নাও থাকতে পারে)।

Lab 20 — Backup Automation

বাস্তব দৃশ্য: TechNova-র একটি প্রকল্পের ডিরেক্টরি প্রতিদিন backup নিতে হবে — তারিখসহ সংকুচিত archive তৈরি, checksum দিয়ে অখণ্ডতা যাচাই, পুরনো backup মুছে ফেলা, আর ব্যর্থ হলে স্পষ্ট error।

উদ্দেশ্য: `tar` + `gzip` দিয়ে archive করা, `sha256sum` দিয়ে যাচাই করা, retention নীতি প্রয়োগ করা ও log লেখা।

পূর্বজ্ঞান: Chapter 15, Lab 09, Lab 11, Lab 13। **পরিবেশ:** `~/labs/lab20`।

ধাপে ধাপে নির্দেশনা

1. পরীক্ষার ডেটা তৈরি করুন:

```
mkdir -p ~/labs/lab20/project ~/labs/lab20/backups
cd ~/labs/lab20
printf 'hello world\n' > project/app.txt
printf 'config v1\n' > project/config.txt
```

2. Backup স্ক্রিপ্ট লিখুন:

```
cat > backup.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
source_dir="$HOME/labs/lab20/project"
backup_dir="$HOME/labs/lab20/backups"
retention_days=7
log_file="$HOME/labs/lab20/backup.log"
stamp="$(date +%Y-%m-%d_%H%M%S)"
archive="$backup_dir/project-$stamp.tar.gz"
if [[ ! -d "$source_dir" ]]; then
  printf '[%s] ERROR: source not found: %s\n' "$(date '+%F %T')" "$source_dir" >>
"$log_file"
  exit 1
fi
mkdir -p -- "$backup_dir"
tar -czf "$archive" -C "$(dirname "$source_dir")" "$(basename "$source_dir")"
sha256sum "$archive" > "$archive.sha256"
if sha256sum -c "$archive.sha256" > /dev/null 2>&1; then
  printf '[%s] OK: created %s\n' "$(date '+%F %T')" "$(basename "$archive")" >>
"$log_file"
else
  printf '[%s] ERROR: checksum failed for %s\n' "$(date '+%F %T')" "$archive" >>
"$log_file"
  exit 1
fi
find "$backup_dir" -maxdepth 1 -name 'project-*.tar.gz' -mtime +"$retention_days" -delete
find "$backup_dir" -maxdepth 1 -name 'project-*.tar.gz.sha256' -mtime +"$retention_days" -
delete
printf 'Backup complete: %s\n' "$archive"
EOF
chmod +x backup.sh
./backup.sh
ls -l backups
cat backup.log
```

3. Restore পরীক্ষা করুন:

```
mkdir -p /tmp/restore-test
tar -xzf backups/project-*.tar.gz -C /tmp/restore-test
ls -R /tmp/restore-test
diff project/app.txt /tmp/restore-test/project/app.txt && echo "Restore OK"
```

4. পুরনো backup মুছে ফেলার পরীক্ষা:

```
touch -d "10 days ago" backups/project-2000-01-01_000000.tar.gz
touch -d "10 days ago" backups/project-2000-01-01_000000.tar.gz.sha256
./backup.sh
ls backups
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`tar -czf "$archive" -C "$(dirname "$source_dir")" "$(basename "$source_dir")"` — `-C` দিয়ে প্রথমে parent ডিরেক্টরিতে গিয়ে তারপর `project` archive করা হয়েছে, যাতে archive-এর ভেতরে সম্পূর্ণ পথ (`/home/nahid/.../project`) না ঢুকে শুধু `project/` থাকে; restore করার সময় এটি নিরাপদ। `sha256sum` archive-এর একটি আঙুলের ছাপ তৈরি করে; পরে `sha256sum -c` দিয়ে যাচাই করা যায় ফাইলটি একটুও বদলেছে কি না — backup

নেওয়া হয়েছে কিন্তু তা নষ্ট, এমন backup-এর চেয়ে backup না থাকা ভালো নয়, তবে যাচাই না করা backup প্রায় একই রকম বিপজ্জনক। `find ... -mtime +7 -delete` ৭ দিনের পুরনো backup মুছে disk বাঁচায় — retention নীতি ছাড়া backup disk ভরে যাবে।

প্রত্যাশিত ফলাফল (illustrative)

```
Backup complete: /home/nahid/labs/lab20/backups/project-2026-11-08_100000.tar.gz
Restore OK
```

দ্বিতীয় চালনার পর ২০০০ সালের ভুয়া পুরনো ফাইল দুটি আর `ls backups`-এ থাকবে না।

যাচাই

- `sha256sum -c backups/*.sha256` বলবে `OK`।
- Restore করা `app.txt` মূল ফাইলের সঙ্গে `diff`-এ মিলবে।
- `cat backup.log`-এ প্রতিটি চালনার `OK` লাইন থাকবে।

TROUBLESHOOTING: `tar: Cowardly refusing to create an empty archive` এলে source ডিরেক্টরি খালি কি না দেখুন। `find -delete` চালানোর আগে সবসময় `-delete` ছাড়া `find` চালিয়ে দেখুন কোন ফাইলগুলো মিলছে — ভুল pattern-এ গুরুত্বপূর্ণ ফাইল মুছে যেতে পারে।

WARNING: Backup একই disk-এ রাখলে disk নষ্ট হলে backup-ও যাবে — গুরুত্বপূর্ণ ডেটার backup আলাদা মেশিনে বা offsite-এ রাখুন। Backup-এ গোপন ফাইল (.env, key) থাকলে archive-টি encrypt করুন বা সুরক্ষিত জায়গায় রাখুন।

Cleanup: `rm -rf /tmp/restore-test; cd ~ && rm -rf ~/labs/lab20`।

Lab 21 — Health Checking

বাস্তব দৃশ্য: TechNova-র staging সার্ভারটি সকালে ধীর মনে হচ্ছে। CPU, memory, disk, সার্ভিসের অবস্থা ও সাম্প্রতিক error log এক জায়গায় দেখে দ্রুত সিদ্ধান্ত নিতে হবে — সমস্যাটি কোথায়।

উদ্দেশ্য: একটি Bash health-check স্ক্রিপ্ট লেখা যা threshold অনুযায়ী OK/WARNING/CRITICAL জানায় ও exit code দেয়।

পূর্বজ্ঞান: Chapter 12, Lab 08, Lab 11। **পরিবেশ:** `~/labs/lab21`।

ধাপে ধাপে নির্দেশনা

1. স্ক্রিপ্টটি লিখুন:

```
mkdir -p ~/labs/lab21 && cd ~/labs/lab21
cat > healthcheck.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
warn_disk=80
crit_disk=90
warn_mem=80
```

```
overall=0
report() {
  local level="$1"
  local message="$2"
  printf '[%s] %s\n' "$level" "$message"
  if [[ "$level" == "CRITICAL" ]]; then
    overall=2
  elif [[ "$level" == "WARNING" && "$overall" -lt 2 ]]; then
    overall=1
  fi
}
disk_usage="$(df --output=pcent / | tail -n 1 | tr -dc '0-9')"
if [[ "$disk_usage" -ge "$crit_disk" ]]; then
  report CRITICAL "Disk usage ${disk_usage}% (threshold ${crit_disk}%)"
elif [[ "$disk_usage" -ge "$warn_disk" ]]; then
  report WARNING "Disk usage ${disk_usage}% (threshold ${warn_disk}%)"
else
  report OK "Disk usage ${disk_usage}%"
fi
mem_total="$(free -m | awk 'NR==2 {print $2}')"
mem_used="$(free -m | awk 'NR==2 {print $3}')"
mem_pct=$((mem_used * 100 / mem_total))
if [[ "$mem_pct" -ge "$warn_mem" ]]; then
  report WARNING "Memory usage ${mem_pct}% (${mem_used}MB/${mem_total}MB)"
else
  report OK "Memory usage ${mem_pct}% (${mem_used}MB/${mem_total}MB)"
fi
load="$(uptime | awk -F'load average:' '{print $2}' | cut -d, -f1 | tr -d ' ')"
report OK "Load average (1 min): $load"
for service in ssh cron; do
  if systemctl is-active --quiet "$service"; then
    report OK "Service $service is active"
  else
    report CRITICAL "Service $service is not active"
  fi
done
if [[ -f /var/log/syslog ]]; then
  errors="$(grep -ci 'error' /var/log/syslog 2>/dev/null || true)"
  report OK "Syslog error lines (today's file): $errors"
fi
printf 'Overall status code: %s\n' "$overall"
exit "$overall"
EOF
chmod +x healthcheck.sh
./healthcheck.sh; echo "exit: $?"
```

2. Threshold বদলে পরীক্ষা করুন:

```
sed 's/warn_disk=80/warn_disk=1/' healthcheck.sh > healthcheck-low.sh
chmod +x healthcheck-low.sh
./healthcheck-low.sh; echo "exit: $?"
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

স্ক্রিপ্টটি Nagios-ধাঁচের exit code ব্যবহার করে: 0 = OK, 1 = WARNING, 2 = CRITICAL — এই প্রথা মনিটরিং টুলগুলো বোঝে, তাই এই স্ক্রিপ্টকে পরে যেকোনো মনিটরিং সিস্টেমে লাগানো যাবে। `report` function প্রতিটি পরীক্ষার ফল ছাপে ও

সবচেয়ে খারাপ অবস্থাটি `overall`-এ ধরে রাখে — একটি CRITICAL থাকলে সামগ্রিক ফল CRITICAL-ই থাকবে, পরের OK তা মুছে দেবে না। `df --output=pcent /` শুধু শতাংশের কলাম দেয়, `tr -dc '0-9'` শুধু সংখ্যা রাখে — এতে তুলনা নিরাপদ হয়। `systemctl is-active --quiet` কোনো আউটপুট না দিয়ে শুধু exit code দেয়, যা `if`-এ সরাসরি ব্যবহার করা যায়। `grep -c`-এর exit code মিল না পেলে 1 হয়, তাই `|| true` দিয়ে `set -e`-এর কারণে স্ক্রিপ্ট থেমে যাওয়া ঠেকানো হয়েছে।

প্রত্যাশিত ফলাফল (illustrative)

```
[OK] Disk usage 44%
[OK] Memory usage 31% (1200MB/3800MB)
[OK] Load average (1 min): 0.10
[OK] Service ssh is active
[OK] Service cron is active
Overall status code: 0
```

Threshold 1% করলে disk লাইনে `[WARNING]` বা `[CRITICAL]` আসবে ও exit code 1 বা 2 হবে।

যাচাই

- স্বাভাবিক চালনায় exit code 0 হবে; threshold কমালে 1 বা 2 হবে।
- `df -h /` ও `free -h`-এর মানের সঙ্গে রিপোর্টের সংখ্যা মিলবে।
- একটি সার্ভিস বন্ধ করে চালালে (পরীক্ষার VM-এ) সেই লাইনে CRITICAL আসবে।

> **TROUBLESHOOTING:** `systemctl` container বা WSL-এ কাজ না করলে সার্ভিস পরীক্ষার অংশটি এড়িয়ে যান — সেটি পরিবেশের সীমা, স্ক্রিপ্টের ভুল নয়। `df` ভিন্ন disk দেখালে `/`-এর বদলে সঠিক mount point ব্যবহার করুন।

> **PRO TIP:** এই স্ক্রিপ্টকে Lab 18-এর cron-এ লাগান: exit code 0 না হলে cron log-এ তা দেখা যাবে, আর পরে alerting যোগ করা যাবে।

Cleanup: `cd ~ && rm -rf ~/labs/lab21`।

Lab 22 — CLI Application (argparse Tool)

বাস্তব দৃশ্য: TechNova-র অপারেটররা একটি কমান্ড-লাইন টুল চায়: `sysinfo --format json --output report.json` চালালে সিস্টেমের তথ্য JSON-এ সংরক্ষিত হবে; `--dry-run` দিলে কিছু না লিখে শুধু দেখাবে কী করত। পেশাদার CLI-র সব বৈশিষ্ট্য — help, validation, dry-run — একসঙ্গে তৈরি করবেন।

উদ্দেশ্য: `argparse` দিয়ে subcommand/option, validation, dry-run mode ও clean exit code-সহ CLI লেখা।

পূর্বজ্ঞান: Chapter 22, Lab 13–14। **পরিবেশ:** `~/labs/lab22`, Python 3.12।

ধাপে ধাপে নির্দেশনা

1. পরিবেশ তৈরি করুন:

```
mkdir -p ~/labs/lab22 && cd ~/labs/lab22
python3 -m venv .venv
source .venv/bin/activate
```

2. CLI টুলটি লিখুন:

```
cat > sysinfo.py <<'EOF'
import argparse
import json
import platform
import shutil
import sys
from pathlib import Path

def collect_info():
    disk = shutil.disk_usage("/")
    return {
        "system": platform.system(),
        "release": platform.release(),
        "machine": platform.machine(),
        "python": platform.python_version(),
        "disk_total_gb": round(disk.total / (1024 ** 3), 2),
        "disk_free_gb": round(disk.free / (1024 ** 3), 2),
    }

def build_parser():
    parser = argparse.ArgumentParser(
        prog="sysinfo",
        description="Collect basic system information and save it as JSON.",
    )
    parser.add_argument(
        "--format",
        choices=["json", "text"],
        default="text",
        help="Output format (default: text).",
    )
    parser.add_argument(
        "--output",
        type=Path,
        default=None,
        help="File to write the report to.",
    )
    parser.add_argument(
        "--dry-run",
        action="store_true",
        help="Show what would be done without writing any file.",
    )
    return parser

def main(argv=None) -> int:
    parser = build_parser()
    args = parser.parse_args(argv)
    if args.format == "json" and args.output is None and not args.dry_run:
        parser.error("--format json requires --output.")
    info = collect_info()
    if args.format == "json":
        text = json.dumps(info, indent=2)
    else:
```

```
        lines = [f"{key}: {value}" for key, value in info.items()]
        text = "\n".join(lines)
    if args.dry_run:
        print("[DRY RUN] Would write the following report:")
        print(text)
        return 0
    if args.output is not None:
        args.output.parent.mkdir(parents=True, exist_ok=True)
        args.output.write_text(text + "\n", encoding="utf-8")
        print(f"Wrote report to {args.output}")
    else:
        print(text)
    return 0

if __name__ == "__main__":
    sys.exit(main())
EOF
python3 sysinfo.py --help
python3 sysinfo.py
python3 sysinfo.py --format json --output report.json
cat report.json
python3 sysinfo.py --format json --dry-run
python3 sysinfo.py --format json; echo "exit: $?"
python3 sysinfo.py --format xml; echo "exit: $?"
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

`argparse` নিজে `--help` তৈরি করে, ভুল option-এ সুন্দর error দেয় ও exit code 2 ফেরত দেয় — হাতে `sys.argv` parse করার চেয়ে এটি অনেক নিরাপদ ও রক্ষণাবেক্ষণযোগ্য। `choices=["json", "text"]` ভুল format আগেই আটকে দেয়। `parser.error(...)` ব্যবহৃত হয়েছে একটি ব্যবসায়িক নিয়মের জন্য: JSON format-এ output ফাইল ছাড়া অর্থ হয় না, তাই সেটি ভুল হিসেবে জানানো হয়েছে। `main(argv=None)` signature-টি পরীক্ষার জন্য গুরুত্বপূর্ণ — test থেকে `main(["--dry-run"])` ডেকে পরীক্ষা করা যায়, command line ছাড়াই (Lab 23-এ দেখা যাবে)। Dry-run mode-এ কোনো ফাইল লেখা হয় না — automation টুলে এটি অপরিহার্য নিরাপত্তা বৈশিষ্ট্য।

প্রত্যাশিত ফলাফল (illustrative)

```
Wrote report to report.json
{
  "system": "Linux",
  "release": "6.8.0-31-generic",
  "machine": "x86_64",
  "python": "3.12.3",
  "disk_total_gb": 50.0,
  "disk_free_gb": 27.0
}
```

`--format json` একা চালালে `error: --format json requires --output.` ও exit code 2 আসবে।

যাচাই

- `python3 sysinfo.py --help`-এ তিনটি option-এর বর্ণনা দেখা যাবে।
- `report.json` বৈধ JSON হবে।
- `--dry-run` চালানোর পর নতুন কোনো ফাইল তৈরি হবে না।

- `--format xml`-এ `invalid choice` error আসবে।

> **TROUBLESHOOTING:** `argparse` error-এ traceback আসে না — এটি স্বাভাবিক, কারণ `parser.error` নিজে সুন্দর বার্তা দিয়ে exit করে। `--output`-এ ভুল পথ দিলে `OSError` আসবে; `parent.mkdir(parents=True, exist_ok=True)` শুধু parent ডিরেক্টরি তৈরি করে, ভুল পথ ঠিক করে না।

> **PRO TIP:** প্রতিটি automation টুলে `--help`, `--dry-run` ও স্পষ্ট exit code রাখুন — ছয় মাস পরে আপনিই এর সবচেয়ে বড় ব্যবহারকারী হবেন।

Cleanup: `deactivate; cd ~ && rm -rf ~/labs/lab22`।

Lab 23 — Automated Testing (pytest)

বাস্তব দৃশ্য: Lab 22-এর CLI টুলটি এখন দলের সবাই ব্যবহার করে। একটি ছোট পরিবর্তনেই সব ভেঙে যেতে পারে — তাই পরিবর্তনের আগে স্বয়ংক্রিয় test লিখবেন, যাতে ভুল ধরা পড়ে deploy-এর আগেই।

উদ্দেশ্য: `pytest` দিয়ে unit test লেখা, fixture ও `tmp_path` ব্যবহার করা, mock দিয়ে বাইরের কমান্ড অনুকরণ করা।

পূর্বজ্ঞান: Chapter 23, Lab 22। **পরিবেশ:** `~/labs/lab23`, Python 3.12, pytest।

ধাপে ধাপে নির্দেশনা

1. পরিবেশ ও Lab 22-এর কোড কপি করুন:

```
mkdir -p ~/labs/lab23 && cd ~/labs/lab23
python3 -m venv .venv
source .venv/bin/activate
pip install pytest
```

2. পরীক্ষার জন্য ছোট একটি মডিউল লিখুন:

```
cat > diskcheck.py <<'EOF'
import shutil

def disk_free_percent(path="/"):
    usage = shutil.disk_usage(path)
    if usage.total == 0:
        raise ValueError("Disk total is zero.")
    return round(usage.free / usage.total * 100, 2)

def status_for(free_percent, warn=20.0, crit=10.0):
    if free_percent < crit:
        return "CRITICAL"
    if free_percent < warn:
        return "WARNING"
    return "OK"
EOF
cat > test_diskcheck.py <<'EOF'
import pytest
from diskcheck import disk_free_percent, status_for
```

```
def test_status_ok():
    assert status_for(50.0) == "OK"

def test_status_warning():
    assert status_for(15.0) == "WARNING"

def test_status_critical():
    assert status_for(5.0) == "CRITICAL"

def test_status_boundary():
    assert status_for(20.0) == "OK"
    assert status_for(10.0) == "WARNING"

def test_disk_free_percent_real():
    value = disk_free_percent("/")
    assert 0 <= value <= 100

def test_disk_free_percent_invalid():
    with pytest.raises(OSError):
        disk_free_percent("/nonexistent-path-xyz")

def test_status_parametrized():
    cases = [(50.0, "OK"), (15.0, "WARNING"), (5.0, "CRITICAL")]
    for value, expected in cases:
        assert status_for(value) == expected
EOF
python3 -m pytest -v
```

3. ইচ্ছাকৃতভাবে একটি ভুল ঢুকিয়ে test ধরতে পারে কি না দেখুন:

```
sed -i 's/< crit/<= crit/' diskcheck.py
python3 -m pytest -v 2>&1 | tail -n 15
sed -i 's/<= crit/< crit/' diskcheck.py
python3 -m pytest -v
```

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

Test-এর নাম `test_` দিয়ে শুরু হয় — pytest এই নাম দেখেই test খুঁজে পায়। প্রতিটি test একটি ছোট দাবি (assertion): `assert status_for(15.0) == "WARNING"` মানে এই input-এ এই output আসতেই হবে, না এলে test ব্যর্থ। Boundary test (`20.0` ঠিক সীমায় OK, `10.0` ঠিক সীমায় WARNING) সবচেয়ে মূল্যবান — ভুলের বেশিরভাগ সীমায় লুকিয়ে থাকে। `pytest.raises(OSError)` পরীক্ষা করে ভুল input-এ সঠিক exception ওঠে কি না — সাফল্যের পাশাপাশি ব্যর্থতার আচরণও পরীক্ষা করতে হয়। `test_disk_free_percent_real` আসল disk পড়ে — এটি একটি integration test; বাকিগুলো pure function-এর unit test, তাই দ্রুত ও নির্ভরযোগ্য। ইচ্ছাকৃত ভুলটি (`<` বদলে `<=`) boundary test ধরে ফেলেছিল — এটাই test-এর প্রমাণ।

প্রত্যাশিত ফলাফল (illustrative)

```
test_diskcheck.py::test_status_ok PASSED
test_diskcheck.py::test_status_warning PASSED
test_diskcheck.py::test_status_critical PASSED
test_diskcheck.py::test_status_boundary PASSED
test_diskcheck.py::test_disk_free_percent_real PASSED
test_diskcheck.py::test_disk_free_percent_invalid PASSED
```

```
test_diskcheck.py::test_status_parametrized PASSED
7 passed in 0.05s
```

যাচাই

- সব test পাস হবে; ভুল ঢোকানোর পর অন্তত একটি test ব্যর্থ হবে।
- `python3 -m pytest -v` প্রতিটি test-এর নামসহ ফল দেখাবে।
- `python3 -m pytest --collect-only` ৭টি test খুঁজে পাবে।

TROUBLESHOOTING: `ModuleNotFoundError: diskcheck` এলে test ফাইল ও মডিউল একই ডিরেক্টরিতে আছে কি না এবং pytest সেই ডিরেক্টরি থেকে চালানো হচ্ছে কি না দেখুন। Test হঠাৎ ধীর হলে দেখুন কোনো test আসল নেটওয়ার্ক বা disk-এর উপর নির্ভর করছে কি না — সেগুলোকে mock দিয়ে আলাদা করুন।

PRO TIP: নতুন feature লেখার আগে তার test লিখুন (test-driven development-এর সরল রূপ) — test-টি প্রথমে ব্যর্থ হবে, feature লেখার পর পাস করবে। এতে feature-টি আসলেই কাজ করে, তা প্রমাণিত হয়।

Cleanup: `deactivate; cd ~ && rm -rf ~/labs/lab23`।

Lab 24 — Complete Toolkit Integration (Git + Bash + Python + Cron)

বাস্তব দৃশ্য: TechNova-র staging সার্ভারে একটি সম্পূর্ণ ছোট toolkit চালু করতে হবে: Bash health-check, Python রিপোর্ট তৈরি, JSON রিপোর্ট, Git-এ সংস্করণ নিয়ন্ত্রণ, আর cron দিয়ে দৈনিক চালনা — বইয়ের সব অংশ একসঙ্গে।

উদ্দেশ্য: আগের lab-গুলোর উপাদানগুলোকে একটি Git repository-তে সাজিয়ে, cron-এ লাগিয়ে একটি end-to-end automation pipeline তৈরি করা।

পূর্বজ্ঞান: Lab 01, 09, 11, 13, 16, 18, 21, 22। **পরিবেশ:** `~/labs/lab24`, Git, Bash, Python 3.12, cron।

ধাপে ধাপে নির্দেশনা

1. Repository ও কাঠামো তৈরি করুন:

```
mkdir -p ~/labs/lab24/toolkit && cd ~/labs/lab24/toolkit
git init
mkdir -p scripts reports logs
cat > .gitignore <<'EOF'
__pycache__/
*.pyc
.venv/
reports/*.json
logs/*.log
EOF
cat > README.md <<'EOF'
# TechNova Mini Toolkit

Daily health check (Bash) + JSON report (Python), run by cron.

Usage:
  ./scripts/healthcheck.sh
```

```
  python3 scripts/report.py --output reports/latest.json
EOF
```

2. Bash health-check লিখুন:

```
cat > scripts/healthcheck.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
disk_usage="$(df --output=pcent / | tail -n 1 | tr -dc '0-9')"
mem_pct="$(free | awk 'NR==2 {printf "%d", $3/$2*100}')"
printf 'disk_usage_percent=%s\n' "$disk_usage"
printf 'memory_usage_percent=%s\n' "$mem_pct"
printf 'load_1min=%s\n' "$(uptime | awk -F'load average:' '{print $2}' | cut -d, -f1 | tr
-d ' ')"
if [[ "$disk_usage" -ge 90 || "$mem_pct" -ge 90 ]]; then
  exit 2
elif [[ "$disk_usage" -ge 80 || "$mem_pct" -ge 80 ]]; then
  exit 1
fi
exit 0
EOF
chmod +x scripts/healthcheck.sh
```

3. Python রিপোর্ট টুল লিখুন:

```
cat > scripts/report.py <<'EOF'
import argparse
import json
import subprocess
import sys
from pathlib import Path

def run_healthcheck():
    result = subprocess.run(
        ["bash", "scripts/healthcheck.sh"],
        capture_output=True,
        text=True,
        timeout=30,
    )
    metrics = {}
    for line in result.stdout.splitlines():
        if "=" in line:
            key, value = line.split("=", 1)
            metrics[key.strip()] = value.strip()
    return result.returncode, metrics

def main(argv=None) -> int:
    parser = argparse.ArgumentParser(description="Build a health report.")
    parser.add_argument("--output", type=Path, required=True)
    parser.add_argument("--dry-run", action="store_true")
    args = parser.parse_args(argv)
    code, metrics = run_healthcheck()
    status = {0: "OK", 1: "WARNING", 2: "CRITICAL"}.get(code, "UNKNOWN")
    report = {"status": status, "exit_code": code, "metrics": metrics}
    if args.dry_run:
        print(json.dumps(report, indent=2))
        return 0
```

```
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(json.dumps(report, indent=2) + "\n", encoding="utf-8")
    print(f"Status: {status}")
    print(f"Wrote {args.output}")
    return 0

if __name__ == "__main__":
    sys.exit(main())
EOF
./scripts/healthcheck.sh; echo "health exit: $?"
python3 scripts/report.py --dry-run
python3 scripts/report.py --output reports/latest.json
cat reports/latest.json
```

4. Git-এ commit করুন ও cron-এ লাগান:

```
git add .
git status
git commit -m "Add integrated health toolkit"
git log --oneline
crontab -l || true
```

Cron-এ পরীক্ষার জন্য (একবার, প্রতি মিনিটে — পরীক্ষার পর সরিয়ে ফেলবেন) এই লাইনটি যোগ করুন, আপনার আসল পথ বসিয়ে:

```
* * * * * /usr/bin/flock -n /home/nahid/labs/lab24/toolkit/run.lock /bin/bash -c 'cd /
home/nahid/labs/lab24/toolkit && python3 scripts/report.py --output reports/latest.json >>
logs/cron.log 2>&1'
```

দুই মিনিট অপেক্ষা করে দেখুন:

```
sleep 130
cat reports/latest.json
cat logs/cron.log
crontab -e
```

পরীক্ষার লাইনটি মুছে সংরক্ষণ করুন।

গুরুত্বপূর্ণ কোডের ব্যাখ্যা

এই toolkit-এ প্রতিটি স্তরের দায়িত্ব আলাদা: Bash শুধু সিস্টেমের তথ্য সংগ্রহ করে ও threshold অনুযায়ী exit code দেয়; Python সেই আউটপুট parse করে JSON রিপোর্ট তৈরি করে — Python সরাসরি shell কমান্ড চালায়নি, Bash স্ক্রিপ্টটিকে argument list দিয়ে ডেকেছে, তাই injection-এর ঝুঁকি নেই। `.gitignore`-এ `reports/*.json` ও `logs/*.log` রাখা হয়েছে — উৎপন্ন ফাইল Git-এ যাবে না, শুধু source code যাবে। Cron লাইনে `flock` duplicate চালনা ঠেকায়, আর `cd ... && python3 ...` নিশ্চিত করে আপেক্ষিক পথ (`scripts/healthcheck.sh`) সঠিক ডিরেক্টরি থেকে চলছে — cron-এর working directory অনিশ্চিত, তাই `cd` অপরিহার্য।

প্রত্যাশিত ফলাফল (illustrative)

```
Status: OK
Wrote reports/latest.json
{
```

```
  "status": "OK",
  "exit_code": 0,
  "metrics": {
    "disk_usage_percent": "44",
    "memory_usage_percent": "31",
    "load_1min": "0.10"
  }
}
```

যাচাই

- `git log --oneline`-এ একটি commit দেখা যাবে; `git status` clean থাকবে (reports ও logs ignore করা হয়েছে)।
- `python3 scripts/report.py --dry-run` কোনো ফাইল লিখবে না।
- Cron চালনার পর `reports/latest.json`-এর সময় বদলাবে ও `logs/cron.log`-এ নতুন লাইন যোগ হবে।
- Threshold বদলে (disk 90%→1%) চালালে status WARNING/CRITICAL হবে।

TROUBLESHOOTING: Cron-এ Python স্ক্রিপ্ট না চললে `logs/cron.log`-এ traceback দেখুন — সাধারণ কারণ `python3`-এর পথ বা `cd` ভুল। `git status`-এ reports দেখা গেলে `.gitignore` commit হয়েছে কি না দেখুন — ignore নিয়ম শুধু untracked ফাইলে কাজ করে।

WARNING: পরীক্ষার cron entry মুছে ফেলতে ভুলবেন না। এই toolkit-টি শেখার জন্য; production-এ লাগানোর আগে Project 03-এর architecture, logging ও test অংশগুলো যোগ করুন।

Cleanup: Crontab থেকে পরীক্ষার লাইন সরান; `cd ~ && rm -rf ~/labs/lab24`।

পরিশিষ্ট B • Independent Assignments — স্বাধীন অনুশীলন

Appendix B: ১৫টি Independent Assignment

এই assignment-গুলো স্বাধীনভাবে করার জন্য। প্রতিটি assignment-এ কাজটি, প্রসঙ্গ ও গ্রহণযোগ্যতার মানদণ্ড (acceptance criteria) দেওয়া হয়েছে — সম্পূর্ণ সমাধান এখানে দেওয়া হয়নি; সমাধান ও ব্যাখ্যা আলাদা Answer Guide-এ আছে। কোনো assignment-এ আটকে গেলে সংশ্লিষ্ট অধ্যায় ও lab-এ ফিরে যান, তারপর আবার চেষ্টা করুন। সময়ের পরামর্শ: Beginner প্রতিটি ১–২ ঘণ্টা, Intermediate ২–৪ ঘণ্টা, Advanced ৪–৮ ঘণ্টা। সব কাজ নিজের পরীক্ষামূলক পরিবেশে করুন; production সিস্টেমে নয়।

> **CONCEPT:** Assignment-এর উদ্দেশ্য নিখুঁত উত্তর নয় — সিদ্ধান্ত নেওয়ার অভ্যাস। প্রতিটি কাজে লিখুন: আপনি কী ধরে নিয়েছেন, কেন এই পদ্ধতি বেছেছেন, আর কী ঝুঁকি রয়ে গেল। পেশাদার engineer-এর কাজের মূল্য এই ব্যাখ্যাতেই।

Assignment 01 — Beginner (Git) — Commit ইতিহাস পড়া ও ব্যাখ্যা করা

প্রসঙ্গ: একটি দলের repository-তে ২০টি commit আছে। একজন নতুন সদস্য হিসেবে আপনাকে ইতিহাস পড়ে বুঝতে হবে প্রকল্পটি কীভাবে এগিয়েছে।

কাজ: Lab 01–02-এর পদ্ধতিতে একটি পরীক্ষামূলক repository তৈরি করুন যাতে অন্তত ৮টি commit, ২টি branch ও ১টি merge থাকে। তারপর `git log --oneline --graph --all`, `git log -p -- <file>` ও `git blame` ব্যবহার করে একটি এক পৃষ্ঠার প্রতিবেদন লিখুন: কোন commit-এ কী বদলেছে, কোন branch কখন main-এ মিশেছে, আর কোন ফাইলটি সবচেয়ে বেশি বদলেছে।

Acceptance criteria: প্রতিবেদনে অন্তত ৫টি commit-এর উদ্দেশ্য সঠিকভাবে ব্যাখ্যা করা হয়েছে; merge commit-টি চিহ্নিত করা হয়েছে; সবচেয়ে বেশি বদলানো ফাইলটি `git log --stat` দিয়ে প্রমাণিত; কোনো অনুমানকে অনুমান হিসেবে লেখা হয়েছে।

Assignment 02 — Beginner (Git) — ভালো Commit বার্তা লেখা

প্রসঙ্গ: একটি repository-র ইতিহাসে `fix`, `update`, `changes` ধরনের অস্পষ্ট বার্তা ভর্তি। আপনাকে ইতিহাসটি পড়ে বুঝতে হবে কোন commit-এ কী হয়েছিল — কিন্তু বার্তা থেকে বোঝা যাচ্ছে না।

কাজ: ৬টি অস্পষ্ট বার্তাসহ একটি পরীক্ষামূলক repository তৈরি করুন। তারপর প্রতিটি commit-এর আসল পরিবর্তন (`git show` দিয়ে) দেখে Conventional Commits ধরনে (যেমন `feat: add disk usage check`, `fix: correct timeout value`) বার্তাগুলো নতুন করে লিখুন — একটি আলাদা নথিতে, মূল ইতিহাস না বদলে। শেষে লিখুন: কোন বার্তাগুলো সবচেয়ে খারাপ ছিল ও কেন।

Acceptance criteria: ৬টি বার্তার সবগুলোই নতুন ধরনে লেখা হয়েছে; প্রতিটি বার্তায় কী বদলেছে ও কেন, তা বোঝা যায়; খারাপ বার্তার সমালোচনায় অন্তত ৩টি সুনির্দিষ্ট কারণ আছে।

Assignment 03 — Beginner (Bash) — Code Reading: একটি স্ক্রিপ্টের আচরণ অনুমান করা

প্রসঙ্গ: নিচের স্ক্রিপ্টটি না চালিয়ে পড়ে বুঝতে হবে এটি কী করে।

```
#!/usr/bin/env bash
set -euo pipefail
count=0
for file in /tmp/data/*.txt; do
  if [[ ! -e "$file" ]]; then
    break
  fi
  if grep -q "ERROR" "$file"; then
    count=$((count + 1))
  fi
done
printf 'Files with ERROR: %d\n' "$count"
```

কাজ: (ক) স্ক্রিপ্টটি লাইন ধরে ধরে ব্যাখ্যা করুন — প্রতিটি লাইন কী করে ও কেন লেখা হয়েছে। (খ) `/tmp/data` ডিরেক্টরি না থাকলে কী হবে? (গ) `/tmp/data`-তে কোনো `.txt` ফাইল না থাকলে কী হবে? (ঘ) ফাইলের নামে space থাকলে কী হবে? প্রতিটি উত্তরের জন্য পরীক্ষা করে প্রমাণ দিন।

Acceptance criteria: চারটি প্রশ্নের উত্তরই সঠিক ও পরীক্ষা দিয়ে প্রমাণিত; ব্যাখ্যায় `set -euo pipefail`, glob, `grep -q` ও exit code-এর ভূমিকা আছে।

Assignment 04 — Beginner (Bash) — নিরাপদ Backup স্ক্রিপ্ট লেখা

প্রসঙ্গ: একটি ছোট দলের সদস্যরা প্রতিদিন একটি ডিরেক্টরি backup নিতে চায়, কিন্তু কেউ command line মনে রাখতে চায় না।

কাজ: একটি Bash স্ক্রিপ্ট লিখুন যা একটি source ডিরেক্টরি ও একটি গন্তব্য ডিরেক্টরি argument হিসেবে নেয়, তারিখসহ নামে `tar.gz` archive তৈরি করে, checksum লেখে ও যাচাই করে। Argument না দিলে usage বার্তা দেবে; source না থাকলে error দিয়ে থামবে; গন্তব্য না থাকলে তৈরি করবে। Lab 20-এর স্ক্রিপ্টকে ভিত্তি করে লিখুন, কিন্তু নিজের ভাষায় সাজান ও প্রতিটি function-এর উদ্দেশ্য একটি README-তে লিখুন।

Acceptance criteria: স্ক্রিপ্টটি `shellcheck`-এ clean; ভুল argument-এ সঠিক exit code (0/1/2) দেয়; checksum যাচাই করে; restore পরীক্ষা করে দেখানো হয়েছে; README-তে ব্যবহারের উদাহরণ আছে।

Assignment 05 — Beginner (Python) — Code Reading: একটি Automation স্ক্রিপ্টের ভুল খুঁজে বের করা

প্রসঙ্গ: নিচের Python কোডটি একটি ডিরেক্টরির সব `.log` ফাইল মুছে ফেলার কথা, কিন্তু এতে তিনটি গুরুতর সমস্যা আছে।

```
import os
import shutil
def clean(folder):
    for name in os.listdir(folder):
        path = os.path.join(folder, name)
        if name.endswith(".log"):
            os.remove(path)
clean("/var/log")
```

কাজ: (ক) তিনটি সমস্যা চিহ্নিত করুন ও প্রতিটির পরিণতি ব্যাখ্যা করুন। (খ) কোডটি নিরাপদভাবে নতুন করে লিখুন: শুধু একটি নির্দিষ্ট পরীক্ষামূলক ডিরেক্টরিতে কাজ করবে, মুছে ফেলার আগে dry-run mode-এ তালিকা দেখাবে, ব্যবহারকারীর

নিশ্চিতকরণ চাইবে, আর প্রতিটি কাজ log করবে। (গ) ব্যাখ্যা করুন কেন `/var/log`-এর মতো সিস্টেম ডিরেক্টরিতে এমন স্ক্রিপ্ট চালানো বিপজ্জনক।

Acceptance criteria: তিনটি সমস্যাই (যেমন: কোনো বয়স পরীক্ষা নেই, কোনো নিশ্চিতকরণ নেই, সিস্টেম ডিরেক্টরিতে চালানো যায়) চিহ্নিত করা হয়েছে; নতুন কোডে dry-run, নিশ্চিতকরণ ও logging আছে; কোডটি শুধু পরীক্ষামূলক ডিরেক্টরিতে চালিয়ে প্রমাণ করা হয়েছে।

Assignment 06 — Intermediate (Git) — Conflict-সহ Collaboration অনুকরণ

প্রসঙ্গ: আপনি ও একজন সহকর্মী (দুটো আলাদা clone হিসেবে অনুকরণ করবেন) একই ফাইলের একই অংশ বদলেছেন। দুজনের কাজই দরকারি।

কাজ: একটি bare repository তৈরি করে (`git init --bare`) সেটিকে remote হিসেবে ব্যবহার করুন। দুটি আলাদা clone-এ একই ফাইলের একই লাইন আলাদাভাবে বদলান, দুজনেই push করার চেষ্টা করুন। দ্বিতীয়জনের push ব্যর্থ হবে — pull/merge করে conflict সমাধান করুন, দুই পাশের যুক্তিসংগত অংশ রেখে, তারপর push করুন। পুরো প্রক্রিয়াটি ধাপে ধাপে একটি নথিতে লিখুন, প্রতিটি ধাপে কোন কমান্ড কী করল তা ব্যাখ্যাসহ।

Acceptance criteria: Bare repository remote হিসেবে কাজ করেছে; push ব্যর্থতার কারণ ব্যাখ্যা করা হয়েছে; conflict সমাধানে দুই পাশের পরিবর্তনই বিবেচনা করা হয়েছে; চূড়ান্ত ফাইলে কোনো marker নেই; নথিতে প্রতিটি ধাপের কমান্ড ও তার ফল আছে।

Assignment 07 — Intermediate (Bash) — Debugging: ভাঙা স্ক্রিপ্ট ঠিক করা

প্রসঙ্গ: একটি deploy স্ক্রিপ্ট মাঝে মাঝে ব্যর্থ হয় — কখনো ঠিক চলে, কখনো ভুল ফাইল কপি করে। কারণটি অস্পষ্ট।

```
#!/usr/bin/env bash
files=$(ls /tmp/build/*.txt)
for f in $files; do
  cp $f /tmp/deploy/
done
echo Done
```

কাজ: (ক) স্ক্রিপ্টটিতে অন্তত ৪টি সমস্যা খুঁজুন (quoting, word splitting, error handling, খালি glob ইত্যাদি)। (খ) ফাইলের নামে space, `/tmp/build` খালি থাকা, ও `cp` ব্যর্থ হওয়া — এই তিন পরিস্থিতিতে পরীক্ষা করে প্রতিটি সমস্যা প্রমাণ করুন। (গ) স্ক্রিপ্টটি নিরাপদভাবে নতুন করে লিখুন ও ShellCheck দিয়ে যাচাই করুন।

Acceptance criteria: ৪টি সমস্যাই চিহ্নিত ও পরীক্ষা দিয়ে প্রমাণিত; নতুন স্ক্রিপ্টে `set -euo pipefail`, quoting ও খালি-ডিরেক্টরি পরীক্ষা আছে; ShellCheck clean; তিনটি পরিস্থিতিতেই নতুন স্ক্রিপ্ট সঠিক আচরণ করে।

Assignment 08 — Intermediate (Bash) — Log বিশ্লেষণ টুল

প্রসঙ্গ: একটি সার্ভারের log ফাইলে প্রতিদিন হাজার হাজার লাইন জমে। অপারেটর চায় একটি কমান্ডে জানতে: মোট request, error-এর সংখ্যা, সবচেয়ে বেশি request করা ৫টি IP, সবচেয়ে ধীর ৫টি URL (যদি log-এ সময় থাকে)।

কাজ: Lab 10-এর ভিত্তিতে একটি পূর্ণাঙ্গ log বিশ্লেষণ স্ক্রিপ্ট লিখুন। এটি argument হিসেবে log ফাইলের পথ নেবে, ফাইল না থাকলে error দেবে, আর একটি সাজানো রিপোর্ট ছাপবে। নিজের তৈরি করা অন্তত ৫০ লাইনের একটি নমুনা log দিয়ে পরীক্ষা করুন ও ফল হাতে গুনে যাচাই করুন। স্ক্রিপ্টে function ব্যবহার করুন ও প্রতিটি function-এর কাজ README-তে লিখুন।

Acceptance criteria: রিপোর্টে ৪টি তথ্যই সঠিক; ৫০ লাইনের নমুনায় হাতে গোনা সংখ্যার সঙ্গে মিলেছে; ভুল ফাইল পথে সঠিক error ও exit code; ShellCheck clean; README-তে ব্যবহারের উদাহরণ আছে।

Assignment 09 — Intermediate (Python) — API থেকে রিপোর্ট তৈরি

প্রসঙ্গ: GitHub API থেকে একটি সংস্থার (organization) কয়েকটি public repository-র তথ্য সংগ্রহ করে একটি সাপ্তাহিক রিপোর্ট তৈরি করতে হবে।

কাজ: Lab 15-এর ভিত্তিতে একটি Python প্রোগ্রাম লিখুন যা একটি YAML configuration ফাইল থেকে repository-র তালিকা পড়ে (`owner/repo` ধরনে), প্রতিটির জন্য GitHub API ডাকে (timeout, status code পরীক্ষা, rate-limit বার্তা সামলে), তথ্য সংগ্রহ করে একটি JSON রিপোর্ট লেখে ও console-এ সারাংশ ছাপে। একটি repository না পাওয়া গেলে প্রোগ্রামটি থামবে না — সেটি রিপোর্টে `not found` হিসেবে লিখে বাকিগুলো চালিয়ে যাবে। Structured logging যোগ করুন।

Acceptance criteria: YAML থেকে অন্তত ৩টি repository পড়া হয়েছে; একটি ভুল নাম দিয়ে not-found আচরণ পরীক্ষা করা হয়েছে; rate-limit (403) বার্তা সামলানো হয়েছে; JSON রিপোর্ট বৈধ; কোনো token কোডে লেখা হয়নি; test-এ অন্তত ২টি function-এর pytest লেখা হয়েছে।

Assignment 10 — Intermediate (Python) — Configuration Validation ডিজাইন

প্রসঙ্গ: একটি automation টুল ভুল configuration পেলে মাঝপথে ব্যর্থ হয় — তখন অর্ধেক কাজ হয়ে গেছে, অর্ধেক হয়নি। সমাধান: কাজ শুরুর আগেই পুরো configuration যাচাই করা।

কাজ: একটি YAML configuration schema ডিজাইন করুন একটি backup টুলের জন্য: source ডিরেক্টরি, গন্তব্য, retention দিন, বাদ দেওয়ার pattern-এর তালিকা। একটি Python validator লিখুন যা প্রতিটি ক্ষেত্রের অস্তিত্ব, ধরন (string/list/int), মানের সীমা (যেমন retention 1–90 দিন) ও পথের অস্তিত্ব পরীক্ষা করে; ভুল থাকলে সব ভুল একসঙ্গে একটি তালিকায় জানাবে (প্রথম ভুলেই থামবে না)। অন্তত ৫টি ভিন্ন ভুল configuration দিয়ে পরীক্ষা করুন।

Acceptance criteria: Schema নথিতে লেখা হয়েছে; validator সব ভুল একসঙ্গে জানায়; ৫টি ভুল configuration-এর প্রতিটিতে সঠিক বার্তা আসে; বৈধ configuration পাস করে; pytest-এ অন্তত ৪টি test আছে।

Assignment 11 — Intermediate (Automation) — Cron + Health Check + Alert

প্রসঙ্গ: একটি সার্ভারের disk ৯০% ছাড়ালে কেউ জানতে পারে না, যতক্ষণ না সার্ভার থেমে যায়।

কাজ: Lab 18 ও Lab 21 একত্রিত করুন: একটি health-check স্ক্রিপ্ট লিখুন যা disk, memory ও একটি গুরুত্বপূর্ণ সার্ভিস পরীক্ষা করে, ফল একটি log ফাইলে লেখে, আর CRITICAL হলে একটি alert ফাইল (`alerts/` ডিরেক্টরিতে timestamp-সহ) তৈরি করে। Cron-এ প্রতি ১৫ মিনিটে চালানোর entry লিখুন (নথিতে, পরীক্ষার জন্য প্রতি মিনিটে চালিয়ে যাচাই করুন)। `flock` ব্যবহার করুন। একটি পরীক্ষা লিখুন যাতে threshold বদলে CRITICAL অবস্থা তৈরি করে alert ফাইল তৈরি হওয়া প্রমাণ করা যায়।

Acceptance criteria: স্বাভাবিক অবস্থায় alert ফাইল তৈরি হয় না; threshold কমালে alert ফাইল তৈরি হয়; log-এ প্রতিটি চালনার রেকর্ড আছে; cron entry সঠিক syntax-এ লেখা; `flock` ব্যবহার করা হয়েছে; পরীক্ষার cron entry শেষে সরানো হয়েছে।

Assignment 12 — Advanced (Git + Python) — Repository স্বাস্থ্য পরীক্ষক

প্রসঙ্গ: একটি দলের অনেকগুলো repository-তে বড় ফাইল ভুল করে commit করা হয়েছে, `.gitignore` নেই, আর commit বার্তা অস্পষ্ট। একটি টুল চাই যা repository পরীক্ষা করে সমস্যা জানাবে।

কাজ: একটি Python CLI টুল লিখুন যা একটি Git repository-র পথ নিয়ে পরীক্ষা করে: (ক) `.gitignore` আছে কি না; (খ) ১ MB-এর বড় ফাইল commit করা হয়েছে কি না (`git ls-files` ও আকার যাচাই); (গ) সর্বশেষ ১০টি commit বার্তার দৈর্ঘ্য (১০ অক্ষরের কম বার্তা সতর্কতা); (ঘ) uncommitted পরিবর্তন আছে কি না। ফল JSON-এ লিখুন ও console-এ সারাংশ ছাপুন। Git কমান্ডগুলো `subprocess`-এ argument list দিয়ে চালান (`shell=True` নয়)। অন্তত ৩টি ভিন্ন পরীক্ষামূলক repository দিয়ে যাচাই করুন ও pytest লিখুন।

Acceptance criteria: ৪টি পরীক্ষাই কাজ করে; বড় ফাইল ও ছোট বার্তা সঠিকভাবে ধরা পড়ে; JSON রিপোর্ট বৈধ; `shell=True` কোথাও ব্যবহার করা হয়নি; pytest-এ অন্তত ৫টি test পাস করে; README আছে।

Assignment 13 — Advanced (Bash + SSH) — বহু সার্ভারে নিরাপদ Remote কাজ

প্রসঙ্গ: TechNova-র ৩টি staging সার্ভারে একই health-check চালাতে হবে। প্রতিটি সার্ভারে আলাদাভাবে login করে চালানো সময়সাপেক্ষ ও ভুলপ্রবণ।

কাজ: একটি Bash স্ক্রিপ্ট লিখুন যা একটি সার্ভার তালিকা ফাইল (প্রতি লাইনে একটি hostname) পড়ে, প্রতিটি সার্ভারে SSH দিয়ে একটি নির্দিষ্ট কমান্ড চালায় (যেমন `df -h / | tail -n 1`), ফল সংগ্রহ করে একটি রিপোর্টে লেখে, আর কোনো সার্ভারে সংযোগ ব্যর্থ হলে সেটি রিপোর্টে `UNREACHABLE` হিসেবে লিখে বাকিগুলো চালিয়ে যায়। Host key যাচাই বন্ধ করবেন না (`StrictHostKeyChecking=no` নিষিদ্ধ)। localhost ও একটি ভুল hostname দিয়ে পরীক্ষা করুন। নিরাপত্তা বিশ্লেষণ লিখুন: এই স্ক্রিপ্টে কী ঝুঁকি আছে ও কীভাবে কমানো যায়।

Acceptance criteria: localhost-এ সফলভাবে চলে; ভুল hostname-এ থামে না, রিপোর্টে UNREACHABLE লেখে; host key যাচাই বন্ধ করা হয়নি; timeout ব্যবহার করা হয়েছে; নিরাপত্তা বিশ্লেষণে অন্তত ৩টি ঝুঁকি ও প্রতিকার আছে; ShellCheck clean।

Assignment 14 — Advanced (Python) — Idempotent Automation ডিজাইন

প্রসঙ্গ: একটি setup স্ক্রিপ্ট দ্বিতীয়বার চালালে duplicate user তৈরি করে, একই লাইন configuration ফাইলে দুবার যোগ করে, আর একই ডিরেক্টরি দুবার তৈরি করার চেষ্টা করে ব্যর্থ হয়।

কাজ: একটি Python টুল লিখুন যা একটি সার্ভার সেটআপের তিনটি কাজ idempotent-ভাবে করে: (ক) একটি ডিরেক্টরি কাঠামো তৈরি (না থাকলে তৈরি, থাকলে কিছু না করা); (খ) একটি configuration ফাইলে একটি লাইন যোগ করা (লাইনটি থাকলে আবার যোগ না করা); (গ) একটি JSON state ফাইলে কাজের রেকর্ড রাখা (কী করা হয়েছে, কখন)। টুলটি দুবার চালালে দ্বিতীয়বার কোনো পরিবর্তন হবে না — এটি পরীক্ষা দিয়ে প্রমাণ করুন (ফাইলের checksum দুবার চালানোর পর একই থাকবে)। Dry-run mode যোগ করুন। Architecture সিদ্ধান্ত নথিতে লিখুন: কেন এই পদ্ধতি বেছে নিয়েছেন, আর কোন ক্ষেত্রে idempotency ভাঙতে পারে।

Acceptance criteria: দুবার চালানোর পর checksum অপরিবর্তিত; dry-run কোনো ফাইল বদলায় না; state ফাইলে প্রতিটি কাজের রেকর্ড আছে; pytest-এ অন্তত ৫টি test; architecture নথিতে অন্তত ৩টি সিদ্ধান্তের যুক্তি আছে।

Assignment 15 — Advanced (Architecture) — TechNova-র Automation নকশা

প্রসঙ্গ: TechNova-র ৫টি সার্ভার, ৩টি পরিবেশ (dev, staging, production), ২ জন operator। এখন প্রতিটি কাজ হাতে করা হয়। আপনাকে একটি automation architecture প্রস্তাব করতে হবে — কোড লেখার আগে নকশা।

কাজ: একটি ৩–৫ পৃষ্ঠার নকশা নথি লিখুন: (ক) কোন কাজ Bash-এ, কোনটি Python-এ করবেন ও কেন (সিদ্ধান্তের যুক্তিসহ); (খ) Git repository কাঠামো (branch strategy, code review নিয়ম); (গ) Configuration ও secret ব্যবস্থাপনা (কোথায় রাখবেন, কোথায় রাখবেন না); (ঘ) Scheduling (cron বনাম systemd timer) ও log ব্যবস্থাপনা; (ঙ) ব্যর্থতা সামলানো (retry, alert, rollback ধারণা); (চ) নিরাপত্তা ঝুঁকি ও প্রতিকার; (ছ) প্রথম ৪ সপ্তাহের বাস্তবায়ন পরিকল্পনা (milestone-সহ)। একটি ছোট proof-of-concept (একটি health-check + একটি cron entry + একটি রিপোর্ট) দিয়ে নকশার একটি অংশ প্রমাণ করুন।

Acceptance criteria: নথিতে ৭টি বিষয়ই আছে; Bash বনাম Python সিদ্ধান্তে অন্তত ৪টি যুক্তি আছে; secret ব্যবস্থাপনায় কোডে গোপন তথ্য না রাখার নিয়ম স্পষ্ট; ৪ সপ্তাহের পরিকল্পনায় milestone ও গ্রহণযোগ্যতার মানদণ্ড আছে; proof-of-concept চলে ও তার ফল নথিতে উল্লেখ আছে।

> **CHECK YOUR UNDERSTANDING:** সব assignment শেষ করার পর নিজেকে জিজ্ঞেস করুন: আমি কি প্রতিটি কাজের জন্য সঠিক টুল বেছেছি? ব্যর্থতা সামলেছি কি? গোপন তথ্য কোথাও ফাঁস করিনি তো? এই তিন প্রশ্নের উত্তর হ্যাঁ হলে আপনি production-এর দিকে এগোচ্ছেন।

পরিশিষ্ট C • Major Projects — প্রধান প্রকল্প

Appendix C: তিনটি Major Project

এই তিনটি প্রকল্প বইয়ের সব শেখা একত্রিত করে। প্রতিটি প্রকল্প একটি বাস্তব পণ্যের মতো করে তৈরি করতে হবে: কাজ করে, পরীক্ষা করা যায়, নথি আছে, আর অন্য কেউ চালাতে পারে। প্রকল্প জমা দেওয়ার আগে নিজের কাজটি এই তালিকায় যাচাই করুন: README আছে কি? ভুল input-এ নিরাপদে ব্যর্থ হয় কি? গোপন তথ্য কোথাও নেই তো? দুবার চালালে একই ফল আসে কি? Test পাস করে কি?

Project 01 — Linux Server Maintenance Toolkit (Bash)

বাস্তব দৃশ্য: TechNova-র operator-রা প্রতিদিন সকালে পাঁচটি সার্ভারে login করে `df`, `free`, `systemctl` ও `journalctl` আলাদাভাবে চালায়। কখনো একটি সার্ভার বাদ পড়ে, কখনো ফল লেখা হয় না। একটি একক Bash toolkit চাই যা এক কমান্ডে পুরো স্বাস্থ্য পরীক্ষা করে একটি পরিষ্কার রিপোর্ট দেবে।

উদ্দেশ্য ও প্রয়োজনীয়তা

Toolkit-টি করবে: CPU inspection (load average ও সবচেয়ে বেশি CPU ব্যবহার করা ৫টি process), memory inspection (মোট, ব্যবহৃত, উপলব্ধ ও শতাংশ), disk monitoring (প্রতিটি mount-এর ব্যবহার, threshold-এর বেশি হলে সতর্কতা), service status (একটি configuration ফাইলে তালিকাভুক্ত সার্ভিসগুলোর অবস্থা), log inspection (সাম্প্রতিক error লাইনগুলোর সারাংশ), ও health reporting (OK/WARNING/CRITICAL সারাংশ ও exit code)।

কারিগরি প্রয়োজনীয়তা

- **Modular functions:** প্রতিটি পরীক্ষা আলাদা function — `check_cpu`, `check_memory`, `check_disk`, `check_services`, `check_logs`, `print_report`। কোনো function ৩০ লাইনের বেশি নয়।
- **Input validation:** Threshold ও configuration ফাইলের পথ argument হিসেবে নেওয়া যাবে; ভুল মান (যেমন disk threshold 150) দিলে স্পষ্ট error দিয়ে থামবে।
- **Safe execution:** `set -euo pipefail`, সব variable quote করা, ফাইলের নামে space সামলানো, `eval` ব্যবহার না করা।
- **Error handling:** কোনো পরীক্ষা ব্যর্থ হলে (যেমন সার্ভিস নেই) toolkit থামবে না — সেই পরীক্ষাটি `UNKNOWN` হিসেবে রিপোর্ট করে বাকিগুলো চালিয়ে যাবে।
- **Clear output:** মানুষের পড়ার মতো রিপোর্ট ও মেশিনের পড়ার মতো `--format json` আউটপুট — দুটোই।
- **Documentation:** README-তে ইনস্টল, ব্যবহার, প্রতিটি option, exit code (0/1/2) ও উদাহরণ।

Repository কাঠামো

```
server-maintenance-toolkit/
  README.md
  toolkit.sh
  lib/
    checks.sh
    report.sh
  config/
    services.conf
  tests/
    test_toolkit.sh
```

```
  docs/
    USAGE.md
```

মূল স্ক্রিপ্টের কাঠামো (উদাহরণ)

```
#!/usr/bin/env bash
set -euo pipefail
script_dir="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
source "$script_dir/lib/checks.sh"
source "$script_dir/lib/report.sh"
warn_disk="${WARN_DISK:-80}"
crit_disk="${CRIT_DISK:-90}"
if [[ "$warn_disk" -ge "$crit_disk" ]]; then
  printf 'Error: WARN_DISK must be less than CRIT_DISK.\n' >&2
  exit 2
fi
```

এই অংশটি দেখায়: library sourcing, পরিবেশ চলক থেকে threshold নেওয়া (default-সহ), ও শুরুতেই validation। বাকি অংশ — প্রতিটি check function ও রিপোর্ট — আপনাকেই লিখতে হবে। `services.conf`-এ প্রতি লাইনে একটি সার্ভিসের নাম থাকবে; খালি লাইন ও `#` দিয়ে শুরু হওয়া মন্তব্য উপেক্ষা করতে হবে।

Milestone

Milestone	কাজ	গ্রহণযোগ্যতা
M1 — ভিত্তি	Repository, README, lib কাঠামো, CPU/memory/disk check	তিনটি check হাতে চালিয়ে সঠিক ফল দেয়
M2 — সার্ভিস ও লগ	Service check, log inspection, threshold validation	বন্ধ সার্ভিস CRITICAL হিসেবে ধরা পড়ে
M3 — রিপোর্ট	Text ও JSON রিপোর্ট, exit code 0/1/2	দুটি format-ই সঠিক, exit code মিলে
M4 — গুণমান	ShellCheck clean, test স্ক্রিপ্ট, USAGE নথি	ShellCheck-এ কোনো সতর্কতা নেই, test পাস করে

গ্রহণযোগ্যতার মানদণ্ড (Acceptance Criteria)

- `./toolkit.sh` চালালে ছয়টি পরীক্ষার সবগুলোর ফলসহ রিপোর্ট আসে।
- `./toolkit.sh --format json` বৈধ JSON দেয়।
- Threshold ভুল দিলে exit code 2 ও স্পষ্ট বার্তা আসে।
- একটি সার্ভিস বন্ধ থাকলে রিপোর্টে CRITICAL ও সামগ্রিক exit code 2 আসে, কিন্তু toolkit থামে না।
- ShellCheck clean; `bash -n` পাস করে; test স্ক্রিপ্টে অন্তত ৫টি পরীক্ষা পাস করে।
- README পড়ে একজন নতুন operator toolkit চালাতে পারে।

মূল্যায়ন Rubric

বিষয়	ওজন	ভালো কাজের লক্ষণ

কার্যকারিতা	35%	ছয়টি পরীক্ষাই সঠিক, threshold কাজ করে
নিরাপত্তা ও নির্ভরযোগ্যতা	25%	Quoting, validation, error handling, ShellCheck clean
কোডের গঠন	20%	Function-ভিত্তিক, পুনঃব্যবহারযোগ্য, পড়ার মতো
নথি ও পরীক্ষা	20%	README সম্পূর্ণ, test আছে ও পাস করে

WARNING: Toolkit-টি শুধু পড়া (read-only) কাজ করবে — কোনো সার্ভিস restart, ফাইল মোছা বা configuration বদলানো যাবে না। পরিবর্তনকারী কাজ আলাদা টুলের বিষয়, আলাদা অনুমতি নিয়ে।

PRO TIP: Lab 21-এর health-check স্ক্রিপ্টকে ভিত্তি করে শুরু করুন, তারপর function-এ ভাগ করুন। প্রথমে একটি check নিখুঁত করুন, তারপর পরেরটি যোগ করুন।

Project 02 — Python Infrastructure Automation Tool (CLI Application)

বাস্তব দৃশ্য: TechNova-র operator-রা চায় একটি একক Python টুল: configuration পড়বে, সিস্টেমের তথ্য সংগ্রহ করবে, একটি HTTP API থেকে তথ্য আনবে, স্বাস্থ্য পরীক্ষা করবে, backup রিপোর্ট তৈরি করবে — আর সবকিছু structured log-এ লিখবে। Bash-এ এই জটিলতা সামলানো কঠিন; Python-এ এটি একটি রক্ষণাবেক্ষণযোগ্য application হবে।

উদ্দেশ্য ও প্রয়োজনীয়তা

টুলটি করবে: YAML configuration পড়া ও যাচাই করা; সিস্টেম তথ্য সংগ্রহ (platform, disk, memory — Lab 14-এর subprocess প্যাটার্নে, `shell=False`); একটি HTTP API ডাকা (timeout, retry, status code সামলে — Lab 15-এর প্যাটার্নে); JSON প্রক্রিয়াকরণ; সিস্টেম স্বাস্থ্য পরীক্ষা (threshold অনুযায়ী OK/WARNING/CRITICAL); backup রিপোর্ট তৈরি (Lab 20-এর ধারণা, Python-এ); structured logging (Lab 17-এর প্যাটার্নে, console + rotating file); ব্যর্থতা নিরাপদে সামলানো (কোনো একটি ধাপ ব্যর্থ হলে বাকিগুলো চলবে, চূড়ান্ত exit code সঠিক হবে)।

কারিগরি প্রয়োজনীয়তা

- **Modular source code:** একটিমাত্র বড় ফাইল নয় — `config.py`, `system_info.py`, `api_client.py`, `health.py`, `backup.py`, `cli.py` আলাদা মডিউল।
- **Virtual environment:** `python3 -m venv .venv`, `requirements.txt`-এ `requests` ও `pyyaml` (সংস্করণসহ), `pip install -r requirements.txt` দিয়ে ইনস্টল।
- **Configuration validation:** কাজ শুরুর আগে পুরো configuration যাচাই (Assignment 10-এর ধারণা) — ভুল থাকলে সব ভুল একসঙ্গে জানাবে।
- **Tests:** pytest-এ অন্তত ১০টি test — configuration validation, health threshold, API error handling (mock দিয়ে), CLI dry-run।
- **README:** ইনস্টল, configuration উদাহরণ, প্রতিটি কমান্ড, exit code, log ফাইলের অবস্থান, সমস্যা সমাধান।

Repository কাঠামো

```
infra-automation-tool/
  README.md
  requirements.txt
```

```
  config.example.yaml
  src/
    __init__.py
    cli.py
    config.py
    system_info.py
    api_client.py
    health.py
    backup.py
    logging_setup.py
  tests/
    test_config.py
    test_health.py
    test_api_client.py
    test_cli.py
  docs/
    ARCHITECTURE.md
```

Configuration উদাহরণ

```
environment: staging
disk_warn_percent: 80
disk_crit_percent: 90
api:
  base_url: https://api.github.com
  timeout_seconds: 10
backup:
  source_dir: /home/nahid/labs/project02-data
  dest_dir: /home/nahid/labs/project02-backups
  retention_days: 7
services:
  - ssh
  - cron
```

Milestone

Milestone	কাজ	গ্রহণযোগ্যতা
M1 — ভিত্তি	venv, requirements, config loading + validation, logging	ভুল config-এ সব ভুল একসঙ্গে জানায়
M2 — সংগ্রহ	System info, API client, health check	API ব্যর্থ হলেও health রিপোর্ট আসে
M3 — রিপোর্ট	Backup রিপোর্ট, JSON আউটপুট, CLI (dry-run সহ)	dry-run কোনো ফাইল লেখে না
M4 — গুণমান	10+ test, README, ARCHITECTURE নথি	সব test পাস করে, নতুন ব্যবহারকারী চালাতে পারে

গ্রহণযোগ্যতার মানদণ্ড (Acceptance Criteria)

- `python -m src.cli --config config.example.yaml --output report.json` চালালে বৈধ JSON রিপোর্ট তৈরি হয়।

- API সার্ভার না পাওয়া গেলে (ভুল URL দিয়ে পরীক্ষা) টুলটি traceback ছাড়া সুন্দর error দিয়ে বাকি ধাপগুলো চালিয়ে যায়।
- ভুল configuration-এ exit code 2 ও সব ভুলের তালিকা আসে।
- `--dry-run` কোনো ফাইল তৈরি করে না।
- কোনো গোপন তথ্য কোডে বা configuration উদাহরণে নেই; token লাগলে পরিবেশ চলক থেকে পড়া হয়।
- pytest-এ অন্তত ১০টি test পাস করে; `python -m pytest` চালিয়ে প্রমাণ করা হয়েছে।

মূল্যায়ন Rubric

বিষয়	ওজন	ভালো কাজের লক্ষণ
কার্যকারিতা	30%	সব ধাপ কাজ করে, ব্যর্থতা সামলানো হয়
কোডের গঠন	25%	মডিউল আলাদা, function ছোট, নাম স্পষ্ট
নিরাপত্তা	20%	shell=False, timeout, গোপন তথ্য নেই, validation আছে
পরীক্ষা ও নথি	25%	10+ test পাস করে, README ও ARCHITECTURE সম্পূর্ণ

> **REAL WORLD:** এই প্রকল্পের কাঠামোটি একটি আসল production টুলের ক্ষুদ্র সংস্করণ। চাকরির সাক্ষাৎকারে এই repository-টি দেখাতে পারবেন — এটি প্রমাণ করে আপনি শুধু স্ক্রিপ্ট লিখতে পারেন না, একটি রক্ষণাবেক্ষণযোগ্য application তৈরি করতে পারেন।

ঐচ্ছিক Advanced Challenge

- API ডাকে exponential backoff-সহ retry যোগ করুন (১ সেকেন্ড, ২ সেকেন্ড, ৪ সেকেন্ড)।
- রিপোর্টে আগের চালনার সঙ্গে তুলনা যোগ করুন (disk ব্যবহার বেড়েছে না কমেছে)।
- GitHub Actions বা অন্য CI-তে pytest চালানোর configuration যোগ করুন।

Project 03 — TechNova Automation Toolkit

বাস্তব দৃশ্য: TechNova এখন বেশ কয়েকটি Linux সার্ভার পরিচালনা করে — web, database ও staging। প্রতিদিনের কাজগুলো — স্বাস্থ্য পরীক্ষা, disk ও memory রিপোর্ট, সার্ভিসের অবস্থা, log সংগ্রহ, backup ও যাচাই, remote কাজ, API থেকে তথ্য আনা, নির্ধারিত চালনা — সব হাতে করা হয়। একজন operator ছুটিতে গেলে কাজ থেমে যায়। TechNova চায় একটি পুনঃব্যবহারযোগ্য toolkit যা এই সব কাজ স্বয়ংক্রিয় করবে, Git দিয়ে পরিচালিত হবে, পরীক্ষা করা যাবে, আর নতুন operator শিখতে পারবে।

Architecture

Toolkit-টি তিনটি স্তরে ভাগ করা:

TechNova Automation Toolkit — Architecture

স্তর ১: Interface — CLI (Python argparse) + Bash wrapper স্ক্রিপ্ট; operator এখানে কমান্ড দেয়

স্তর ২: Core — health, backup, logs, remote, api মডিউল (Python); প্রতিটি মডিউল একটিই দায়িত্ব পালন করে

স্তর ৩: Execution — subprocess (shell=False) দিয়ে Linux কমান্ড; SSH দিয়ে remote সার্ভার; requests দিয়ে API

পাশে: Configuration (YAML + পরিবেশ চলক), Logging (structured, rotating file), Scheduling (cron + flock), Storage (JSON রিপোর্ট, backup archive)

প্রবাহ: CLI → Configuration validation → Core মডিউল → Execution → JSON রিপোর্ট + Log → Exit code (0/1/2)

এই ভাগের যুক্তি: Interface বদলালে Core বদলাতে হয় না; একটি মডিউল ভাঙলে বাকিগুলো চলে; প্রতিটি মডিউল আলাদাভাবে পরীক্ষা করা যায়। Bash শুধু সেখানে ব্যবহার করা হবে যেখানে তা স্বাভাবিক — ছোট system check ও cron wrapper; জটিল যুক্তি, API ও রিপোর্ট Python-এ।

Requirements

কার্যকরী প্রয়োজনীয়তা

- **Git-based source management:** পুরো toolkit একটি Git repository-তে; `main` branch সুরক্ষিত (সরাসরি commit নয়, pull request ও review-এর মাধ্যমে পরিবর্তন); প্রতিটি release-এ tag (যেমন `v1.0.0`); Semantic Versioning।
- **Server health monitoring:** CPU load, memory, disk — প্রতিটি সার্ভারে, threshold-সহ।
- **Disk and memory reports:** দৈনিক JSON রিপোর্ট, আগের দিনের সঙ্গে তুলনা।
- **Service status checks:** Configuration-এ তালিকাভুক্ত সার্ভিসগুলোর অবস্থা।
- **Log collection:** প্রতিটি সার্ভারের সাম্প্রতিক error লাইন সংগ্রহ করে কেন্দ্রীয় রিপোর্টে।
- **Backup and verification:** নির্দিষ্ট ডিরেক্টরির backup, checksum যাচাই, retention নীতি।
- **Remote SSH operations:** Key-based SSH দিয়ে একাধিক সার্ভারে একই পরীক্ষা চালানো; host key যাচাই বন্ধ করা যাবে না।
- **API integration:** অন্তত একটি public API থেকে তথ্য সংগ্রহ (যেমন GitHub repository তথ্য), timeout ও rate-limit সামলে।
- **Scheduled tasks:** cron দিয়ে দৈনিক health-check ও সাপ্তাহিক backup; `flock` দিয়ে duplicate চালনা ঠেকানো।
- **Error handling:** একটি সার্ভার বা ধাপ ব্যর্থ হলে বাকিগুলো চলবে; চূড়ান্ত রিপোর্টে ব্যর্থতা স্পষ্ট থাকবে।
- **Structured logging:** timestamp, level, মডিউলের নামসহ log; console-এ সংক্ষিপ্ত, ফাইলে বিস্তারিত; rotating file।
- **Automated tests:** pytest-এ অন্তত ১৫টি test; mock দিয়ে SSH ও API অনুকরণ।
- **Documentation:** README, ARCHITECTURE, RUNBOOK (সমস্যা হলে কী করতে হবে), CHANGELOG।

অকার্যকরী প্রয়োজনীয়তা

- **নিরাপত্তা:** কোডে কোনো গোপন তথ্য নেই; secret পরিবেশ চলক থেকে আসে; `shell=True` নিষিদ্ধ; সব input যাচাই করা হয়; log-এ গোপন তথ্য লেখা হয় না।

- **Idempotency:** একই কমান্ড দুবার চালালে দ্বিতীয়বার কোনো ক্ষতিকর পরিবর্তন হয় না।
- **পর্যবেক্ষণযোগ্যতা:** প্রতিটি চালনার exit code, log ও JSON রিপোর্ট থেকে বোঝা যায় কী হয়েছে।
- **রক্ষণাবেক্ষণযোগ্যতা:** নতুন সার্ভার যোগ করা মানে শুধু configuration ফাইলে একটি entry যোগ করা, কোড বদলানো নয়।

Repository কাঠামো

```
technova-automation-toolkit/
  README.md
  ARCHITECTURE.md
  RUNBOOK.md
  CHANGELOG.md
  requirements.txt
  config/
    config.example.yaml
    servers.example.yaml
    services.conf
  src/
    __init__.py
    cli.py
    config.py
    health.py
    backup.py
    logs_collector.py
    remote.py
    api_client.py
    reporting.py
    logging_setup.py
  scripts/
    healthcheck.sh
    install-cron.sh
  tests/
    test_config.py
    test_health.py
    test_backup.py
    test_remote.py
    test_api_client.py
    test_cli.py
  docs/
    OPERATOR-GUIDE.md
```

Milestones (৮ সপ্তাহের পরিকল্পনা)

সপ্তাহ	Milestone	কাজ	গ্রহণযোগ্যতা
১	M1 — ভিত্তি	Repository, branch নিয়ম, config schema, logging, CLI কঙ্কাল	CLI --help চলে; ভুল config-এ সব ভুল জানায়
২–৩	M2 — Core: health ও backup	Health check, disk/memory রিপোর্ট, backup + checksum + retention	দুটি সার্ভারে (localhost + VM) health রিপোর্ট আসে; backup restore পরীক্ষা পাস করে
৪	M3 — Remote ও logs	SSH remote execution, log collection	ভুল hostname-এ থামে না; UNREACHABLE রিপোর্টে আসে

৫	M4 — API ও reporting	API client, JSON রিপোর্ট, আগের দিনের সঙ্গে তুলনা	API ব্যর্থ হলেও রিপোর্ট আসে; তুলনা সঠিক
৬	M5 — Scheduling	Cron entry, flock, install-cron.sh, log rotation	দুই দিন cron-এ চলে; duplicate চালনা হয় না
৭	M6 — Testing	15+ pytest, mock-সহ, ShellCheck, নথি সম্পূর্ণ	সব test পাস করে; নতুন operator README পড়ে চালাতে পারে
৮	M7 — Release	Code review, v1.0.0 tag, CHANGELOG, demo	Review-এর মন্তব্য ঠিক করা হয়েছে; tag দেওয়া হয়েছে

Deliverables — যা জমা দিতে হবে

1. Git repository (GitHub-এ public বা private — link-সহ), যেখানে সম্পূর্ণ ইতিহাস দেখা যায় (অন্তত ১৫টি অর্থপূর্ণ commit, feature branch-এ কাজ, pull request-এর মাধ্যমে merge)।
2. কাজ করা toolkit — README অনুযায়ী ইনস্টল করে চালানো যায়।
3. একটি নমুনা দৈনিক রিপোর্ট (JSON) ও একটি সাপ্তাহিক backup রিপোর্ট।
4. Test রিপোর্ট — `python -m pytest -v`-এর আউটপুট।
5. নথি: README, ARCHITECTURE, RUNBOOK, CHANGELOG, OPERATOR-GUIDE।
6. একটি ১০ মিনিটের demo: toolkit চালানো, একটি সার্ভার ব্যর্থ হওয়ার অনুকরণ, ও রিপোর্ট দেখানো (ভিডিও বা লিখিত walkthrough)।

গ্রহণযোগ্যতার মানদণ্ড (Acceptance Criteria)

- Toolkit-টি অন্তত দুটি সার্ভারে (localhost + একটি VM বা দ্বিতীয় মেশিন) চলে।
- একটি সার্ভার unreachable হলে বাকিগুলোর রিপোর্ট আসে ও চূড়ান্ত exit code সঠিক হয়।
- Backup নেওয়া যায়, checksum যাচাই পাস করে, restore পরীক্ষা সফল হয়।
- Cron-এ দুই দিন চলে; log-এ প্রতিটি চালনার রেকর্ড আছে; duplicate চালনা হয় না।
- কোডে কোনো গোপন তথ্য, `shell=True` বা `StrictHostKeyChecking=no` নেই।
- pytest-এ অন্তত ১৫টি test পাস করে; ShellCheck clean।
- নতুন সার্ভার যোগ করা শুধু configuration বদলে করা যায় — প্রমাণসহ।
- RUNBOOK পড়ে একজন নতুন operator একটি সাধারণ সমস্যা (যেমন disk full) সামলাতে পারে।

মূল্যায়ন Rubric

বিষয়	ওজন	ভালো কাজের লক্ষণ
Architecture ও নকশা	20%	স্তর আলাদা, সিদ্ধান্তের যুক্তি নথিতে আছে
কার্যকারিতা	25%	সব প্রয়োজনীয়তা কাজ করে, দুটি সার্ভারে প্রমাণিত
নিরাপত্তা ও নির্ভরযোগ্যতা	20%	গোপন তথ্য নেই, ব্যর্থতা সামলানো হয়, idempotent
পরীক্ষা	15%	15+ test, mock ব্যবহার, সব পাস করে

নথি ও Git workflow	20%	নথি সম্পূর্ণ, branch/PR ইতিহাস অর্থপূর্ণ, release tag আছে

> **WARNING:** এই প্রকল্পে আসল production সার্ভার ব্যবহার করবেন না। সব পরীক্ষা নিজের VM বা staging পরিবেশে করুন। SSH key-তে passphrase দিন, আর private key কখনো repository-তে commit করবেন না — `.gitignore`-এ `*.pem`, `id_*` ও `.env` রাখুন।

ঐচ্ছিক Advanced Challenge

- systemd timer দিয়ে cron-এর বিকল্প বাস্তবায়ন করুন ও দুটির তুলনা লিখুন।
- রিপোর্টগুলো একটি সহজ HTML ড্যাশবোর্ডে দেখান (শুধু স্থানীয়ভাবে, কোনো সার্ভার ছাড়া)।
- Backup-এ encryption যোগ করুন (যেমন `gpg` দিয়ে) ও key ব্যবস্থাপনা নথিতে লিখুন।
- Toolkit-কে একটি Python package হিসেবে সাজান (`pyproject.toml` সহ) যাতে `pip install` করে চালানো যায়।
- একটি incident অনুকরণ করুন: disk পূর্ণ হওয়ার ভান করে RUNBOOK অনুযায়ী সমস্যা সমাধান করুন ও সময় মাপুন।

> **CHECK YOUR UNDERSTANDING:** তিনটি প্রকল্প শেষ করার পর আপনার কাছে থাকবে: একটি Bash toolkit, একটি Python CLI application, আর একটি সম্পূর্ণ automation system — এই তিনটি মিলেই একটি DevOps engineer-এর portfolio-র ভিত্তি। প্রতিটি প্রকল্পের repository link আপনার CV-তে যোগ করতে পারেন।